LatentSQL: Internalising a Multi-Agent Text-to-SQL Pipeline into the Latent Space of a Single Small Language Model

A step-by-step derivation, from first principles to the code

Abstract

Large language models solve Text-to-SQL well but are expensive to run. Small language models (SLMs) are cheap, but individually lack the reasoning capacity for hard queries. The standard remedy is to decompose the task across a *multi-agent system* — retrieval, schema linking, classification, candidate generation, fixing, verification — which restores accuracy at the cost of many model calls, tens of thousands of generated tokens, extra training and coordination machinery, and error propagation between stages.

LatentSQL removes the coordination layer without removing the reasoning. It trains *one* SLM to run the entire pipeline inside its own activation space: each pipeline role is replaced by a small number of *continuous thoughts* — hidden-state vectors fed back into the model as input embeddings, emitting no tokens at all — and only the final SQL query is ever decoded. The architecture follows **IMAD** (Yi, Mueller and Lee, ACL 2026, "Latent Agents: A Post-Training Procedure for Internalized Multi-Agent Debate"), the latent mechanism follows **Coconut** (Hao et al., "Training Large Language Models to Reason in a Continuous Latent Space"), and the role chain follows **CHESS** (Talaei et al., 2024).

This document explains that system from the very beginning, assuming no prior knowledge, and is organised strictly from the lowest level to the highest.

Part I states the problem from nothing. Part II descends to the smallest object in the system — one token, one embedding, one hidden state — and shows exactly where text is created. Part III proves, with the Data Processing Inequality and an entropy ceiling, that writing reasoning out as text is a strictly lossy channel, and computes the size of the loss with this project's own numbers. Part IV builds the continuous thought, the alignment map, and the latent working memory. Part V turns a six-stage agent pipeline into a chain of latent slots. Part VI gives the activation-space theory that makes this possible at all, including a proof that difference-in-means is the optimal linear probe under isotropic noise. Part VII derives the training loss exactly as the code computes it. Part VIII proves a small, uncomfortable fact about Adam that turns out to explain a real failure. Part IX is a complete worked diagnosis of that failure — the stage-4 collapse — and its repair. Part X adds reinforcement learning. Part XI asks what that reinforcement stage does to the agent-subspace geometry Part VI established — proving a structural limit in this project's own GRPO implementation and weighing it against four newly-read papers on latent-reasoning GRPO and persona-vector geometry. Part XII maps every equation to the exact line of code that implements it. Part XIII reports the measured results, including where the method fails. Part XIV lists what is not established. Part XV collects every formula in one place.

Every symbol is defined before it is used. Every proof is given in full. Every number reported is one this project actually measured, and where two explanations are consistent with a measurement, both are stated.

Contents

I The problem, stated from nothing **7**

1 What a Text-to-SQL system does **7**
1.1 How we decide whether the output is right 7
1.2 The two benchmarks 8

2 Why one small model is not enough **8**

3 What a multi-agent pipeline does, and what it costs **8**

4 The idea of this project **9**

5 The formal problem **9**

II The lowest level: one token, one hidden state **10**

6 Step 1: the token and its embedding **10**

7 Step 2: the transformer as a function on embeddings **10**

8 Step 3: the language-model head — the only place text is created **11**

9 Step 4: what generating text actually costs **11**

III The decode bottleneck, made precise **12**

10 Three ways a reasoning step can hand off to the next **12**

11 Background: entropy and mutual information **12**

12 The setup, matching what the code and the baselines actually do **12**

13 Theorem: text handoff cannot beat its own token count **13**
13.1 Putting our numbers in 14
13.2 What this proves, and — importantly — what it does not 14

14 The second, different argument: superposition and search **15**

IV The continuous thought **16**

15 Step 1: latent mode **16**

16 Step 2: the alignment map **16**
16.1 Construction 1: identity (Coconut's own choice) 16
16.2 Construction 2: linear 17

16.3 Construction 3: soft embedding (the default) 17

17 Step 3: latent working memory (the KV cache) 20
17.1 A subtlety the code had to solve: cache mutation under checkpointing 20

18 Step 4: the cost accounting 21

19 Is this beam search? Is it tree search? A precise comparison 21
19.1 Before any formal definition: the exact same numbers, two ways 21

V From a multi-agent pipeline to a chain of latent slots 24

20 The role chain 24

21 The tag vocabulary, and why it is load-bearing 24

22 The sequence, written out 25

23 The curriculum: stage k 25
23.1 How the curriculum is scheduled in practice 26

VI Activation space: why any of this can work 27

24 The residual stream and the linear representation hypothesis 27

25 Difference-in-means, and a proof that it is the right direction 27

26 Steering: using the direction 28

27 IMAD's finding, and what it licenses 29

28 What this does and does not establish for LatentSQL 29

VII The training objective 30

29 Cross-entropy, from scratch 30

30 The COCONUT sequence loss, exactly as the code computes it 30
30.1 Why the latent positions get gradients anyway 31

31 The SQL-readiness probe 31
31.1 A degeneracy: why $j^\star = k$ does nothing 32
31.2 Why $j^\star = k - 1$ is not degenerate 32

VIII The optimiser, and one uncomfortable fact about Adam 34

32 What is actually being trained: LoRA 34

33 Adam, derived **34**
33.1 The normalisation, and what it costs 34

34 Gradient accumulation, and how few steps there really are **35**

IX A worked failure: the stage-4 collapse **37**

35 The symptom **37**

36 What the failures actually looked like **37**

37 Mechanism 1: the objective is saturated by stage 3 **38**

38 Mechanism 2: Adam steps into the noise at full size **38**

39 Mechanism 3: where the drift lands **39**

40 The bonus finding: the fix that was already configured did nothing **39**

41 The repair **39**

42 Result of the retrain **40**

43 Training loss during the retrain, and the attribution caveat **40**

44 Two infrastructure failures worth recording **41**

X Reinforcement learning: internalisation with GRPO **42**

45 Group Relative Policy Optimisation, from the policy-gradient theorem **42**

46 The clipped surrogate and the KL penalty **43**

47 IMAD's reward, specialised to SQL execution **43**
47.1 Dense partial credit, for when GRPO would otherwise learn nothing 43
47.2 Brevity, for when a group is entirely correct 44
47.3 Spurious correctness: when a group is entirely correct, but shouldn't be 44

48 Latent rollouts **46**

49 What GRPO measurably did **46**

XI Does GRPO sharpen the agent subspaces, or not? **48**

50 The one fact everything else follows from **48**

51 A die you roll versus a die you average **48**
51.1 Turning that into a one-line proof 49

52 Two published fixes, and their actual formulas **50**
52.1 Fix 1 — roll inside the formula (Soft-GRPO / Latent-GRPO) 50
52.2 Fix 2 — roll the weights instead of the output (Dropout-GRPO) 50

53 Neither fix is in this project's code **51**

54 So why did our own run still move the numbers? **51**

55 A separate question: did SFT even need to *create* the subspaces? **51**

56 Summary **52**

57 A sibling project, and a genuinely different fix to a shared problem: SERL-SQL **52**
57.1 The same coarse-credit problem, independently named 52
57.2 Two different fixes to the same complaint 53
57.3 A speculative bridge (unimplemented, stated as such) 53
57.4 The one architectural difference that is not open to interpretation 54

XII Complete map from equation to code **55**

58 The end-to-end pipeline, as five scripts **55**

XIII Results, reported honestly, including where the method loses **57**

59 The two tests that actually define success **57**
59.1 An earlier, text-mode arm: Test A never passed 57

60 Test B on the earlier arm, before the architectural change **57**

61 The CHESS-step SQL-direct curriculum: the number that matters **58**

62 Cost **58**
62.1 Speculative decoding: measured, and a dead end on this path 58

63 Smoke-test coverage (verifies mechanism, not accuracy) **59**

XIV What is not established **60**

64 Required ablations, not yet run **60**

65 The attribution gap in the stage-4 repair **60**

66 What Part VI does not establish for this system **60**

67 What Part XI does not establish **60**

68 What Part III's capacity argument does not establish **61**

69 What Part X does not establish **61**

70 Infrastructure and scale limits **61**

XV Every formula in one place **62**

71 The one-paragraph version **63**

Part I
The problem, stated from nothing

1 What a Text-to-SQL system does

A **database** is a collection of tables. A **schema** is the description of those tables: their names, their columns, and the type of each column. Here is a tiny one, written in the language databases themselves use:

```
CREATE TABLE museum   (museum_id INT, name TEXT, opened INT);
CREATE TABLE visitor  (visitor_id INT, name TEXT, age INT);
CREATE TABLE visit    (museum_id INT, visitor_id INT, spent REAL);
```

A **query** is a program in the language SQL that asks a question of those tables and returns a small table of numbers or strings as its answer:

```
SELECT name FROM visitor WHERE age > 30;
```

Text-to-SQL is the task of writing that program automatically. The input is a question in English together with the schema; the output is a SQL query. Formally, we want a function

$$f : (q, S) \longmapsto y,$$

where q is the question, S is the schema, and y is a SQL query.

1.1 How we decide whether the output is right

We do *not* compare the generated query to the gold query as strings. Two different SQL programs very often compute exactly the same answer, and a string comparison would call one of them wrong for no reason.

Definition 1 (Execution accuracy, EX)**.** Let y be the predicted query and $y^\star$ the gold query, both run against the same database D. Write $\mathrm{exec}(y, D)$ for the table of results returned. Then the example is scored

$$\mathrm{EX}(y, y^\star, D) = \begin{cases} 1 & \text{if } \mathrm{exec}(y, D) = \mathrm{exec}(y^\star, D), \\ 0 & \text{otherwise (including if } y \text{ fails to parse or errors).} \end{cases}$$

Over a test set of n examples, **execution accuracy** is the mean of this quantity, reported as a percentage.

Remark 2. Comparison of two result tables needs one subtlety: if the gold query contains an `ORDER BY` clause, row order is part of the answer and must match; if it does not, the two tables are compared as multisets of rows. This is what `latentsql/db/execute.py` does, via `gold_is_ordered` and `result_match`.

> Every accuracy number in this document is EX in the sense of Definition 1: the query was actually executed against the actual database, and the returned rows were compared. Nothing is scored by string similarity.

1.2 The two benchmarks

We use a 50/50 mixture of two standard datasets.

- **Spider**: many small databases, questions that are mostly single-table or simple joins. Easier.
- **BIRD**: large, messy, real-world databases, questions that need external "evidence" hints and often numeric reasoning. Much harder.

Because the two differ so much in difficulty, a single averaged number hides what happened. Every result table below therefore reports EX overall *and* EX on each half separately.

2 Why one small model is not enough

A **language model** is a function that, given a sequence of text, produces a probability distribution over the next token; generating text means sampling from that distribution repeatedly. A **small language model (SLM)** here means one with roughly 7 billion parameters or fewer — specifically `Qwen2.5-Coder-7B-Instruct`, which fits on a single 24 GB consumer GPU.

Ask such a model a hard BIRD question directly, in one shot, and it frequently fails. The reason is not that it lacks knowledge of SQL. It is that a hard question requires several distinct *kinds* of work in sequence:

1. Find which literal values in the question actually occur in the database ("Los Angeles" might be stored as `LA`).
2. Decide which of 80 tables and 600 columns are relevant at all.
3. Decide whether the query is simple, needs a join, or needs a nested subquery.
4. Write a candidate query.
5. Run it, see the error, and fix it.
6. Check that the fixed query actually answers the question asked.

Each of these is easy in isolation. Doing all six inside a single forward pass, with no intermediate scratch space, is not.

3 What a multi-agent pipeline does, and what it costs

The established fix is to give the model scratch space by *decomposing* the task. Instead of one call, we make several, each with its own prompt, each handling one of the six jobs above, each passing its output as text to the next. This is what DIN-SQL, DAIL-SQL, CHESS and CHASE-SQL all do in different arrangements. CHESS in particular names four specialised agents — Information Retriever, Schema Selector, Candidate Generator, Unit Tester — and it is CHESS's decomposition we adopt.

> **The multi-agent system works.** It is not a straw man. It genuinely raises accuracy, and it is the right baseline to beat.

But it costs four separate things:

1. **Tokens.** Every intermediate result must be *written out* in words so the next stage can read it. In this project's own measurements, a fully verbalised run averages 2922.8 total tokens per question, of which 1339.3 are generated.

2. **Latency and money.** Each stage is a separate forward pass over a long prompt.

3. **Machinery.** Someone must write, tune, and maintain the orchestration: the prompts, the routing, the retry logic.

4. **Error propagation.** If schema linking drops the one column the answer needs, no later stage can recover it. The mistake is now baked into the text that everything downstream reads.

4 The idea of this project

Keep the decomposition. Delete the text.

Train one small model so that each pipeline stage happens as a handful of *internal vector states* rather than as written words, and only the final SQL query is ever decoded into tokens.

If this works, all four costs fall at once: the intermediate tokens disappear (they were never generated), the extra forward passes collapse into one sequence, the orchestration is gone (it is inside the weights), and error propagation is softened because a vector can carry an uncertain, partially formed hypothesis in a way a committed sentence cannot — a point made precise in Section 14.

5 The formal problem

Definition 3 (The object we are building)**.** Let π_θ be a language model with parameters θ. We want to choose θ so that, on a held-out set $\{(q_i, S_i, y_i^\star, D_i)\}_{i=1}^n$, the model's output y_i maximises

$$\frac{1}{n}\sum_{i=1}^{n} \mathrm{EX}(y_i, y_i^\star, D_i)$$

subject to the constraint that the number of tokens decoded per question is small — ideally just the SQL query itself, roughly 60 tokens.

Note carefully what is *not* the goal. We are not trying to beat a frontier LLM. We are trying to answer two much sharper questions:

Test A. Does a single SLM that runs the pipeline *latently* beat the same SLM trained to run it *in text*?

Test B. Does it match the explicit multi-agent system at the same model size, at a fraction of the tokens?

Everything below is (i) building the latent mechanism, (ii) proving what it can and cannot buy, (iii) training it, and (iv) measuring it.

Part II
The lowest level: one token, one hidden state

Nothing in this part is specific to our method. It is the standard machinery of a transformer language model, written out in the exact notation the rest of the document uses. If you already know it, you can skim to Section 9, which is where the thesis actually begins.

6 Step 1: the token and its embedding

Text is first cut into **tokens** — pieces roughly the size of a common word or a fragment of a rare one. The set of all possible tokens is the **vocabulary** V; each token is identified by an integer index. For our model,

$$|V| = 152{,}064.$$

A model cannot do arithmetic on an integer index, so each token x is mapped to a vector by an **embedding function**

$$e : V \to \mathbb{R}^d, \qquad d = 3584 \text{ for our model.}$$

Concretely e is a lookup in a matrix $W_{\text{in}} \in \mathbb{R}^{|V| \times d}$: row x of that matrix *is* $e(x)$. The number d is called the **hidden dimension**.

Example 4. If the token `"SELECT"` has index 4416, then $e(\texttt{"SELECT"})$ is row 4416 of W_{in}: a specific list of 3584 real numbers.

> **The single most important fact in this document:** the model does not actually consume tokens. It consumes *vectors in* $\mathbb{R}^d$. Tokens are merely the usual way of producing those vectors. Nothing forces the input vector at a position to be a row of W_{in}.

7 Step 2: the transformer as a function on embeddings

Given a sequence of tokens $x = (x_1, \ldots, x_T)$, form the matrix of their embeddings up to position t:

$$E_t = \big[\, e(x_1),\ e(x_2),\ \ldots,\ e(x_t) \,\big] \in \mathbb{R}^{t \times d}.$$

The transformer is a function that maps this to a matrix of the same shape:

$$H_t = \text{Transformer}(E_t) \in \mathbb{R}^{t \times d}. \tag{2.1}$$

Row s of H_t is the model's internal representation of position s after it has looked at positions 1 through s. We write

$$h_t = H_t[t, :] \in \mathbb{R}^d$$

for the last row: the **last hidden state**, the representation of the most recent position. This is the vector the whole method is built on.

Remark 5 (causality)**.** The transformer is *causal*: row s depends only on positions $\leq s$. That is what makes it possible to compute $h_1, h_2, \ldots$ one at a time and reuse the earlier work — see Section 17.

Remark 6 (the final normalisation)**.** h_t as we use it is taken *after* the model's final normalisation layer, so its magnitude is controlled — a typical entry is order 1, not order 100. Coconut makes exactly this observation, and it matters: a raw, un-normalised state fed back as an input would be far outside the range the model's first layer expects.

8 Step 3: the language-model head — the only place text is created

To turn h_t into a next token, the model applies one more matrix, the **output head** $W_{\text{out}} \in \mathbb{R}^{|V|\times d}$, and then a softmax:

$$\ell_t = W_{\text{out}}\, h_t \in \mathbb{R}^{|V|}, \qquad p(x_{t+1} = v \mid x_{\le t}) = \text{softmax}(\ell_t)_v = \frac{\exp(\ell_{t,v})}{\sum_{u\in V}\exp(\ell_{t,u})}. \tag{2.2}$$

The entries of ℓ_t are called **logits**. The softmax turns them into a probability distribution: every entry lands in $(0,1)$ and they sum to 1.

> Equation (2.2) is the *only* step in the entire model where a continuous internal state becomes discrete text. Everything before it is vectors.

9 Step 4: what generating text actually costs

Ordinary generation — what chain-of-thought does, and what a multi-agent pipeline does between its stages — is the following loop:

$$h_t \xrightarrow{W_{\text{out}},\ \text{softmax}} p(\cdot) \xrightarrow{\text{sample or argmax}} x_{t+1} \xrightarrow{e(\cdot)} e(x_{t+1}) = W_{\text{in}}[x_{t+1},:] \longrightarrow \text{next step's input.} \tag{2.3}$$

Look at the shapes along that path:

$$\underbrace{\mathbb{R}^{3584}}_{h_t} \longrightarrow \underbrace{\Delta^{|V|-1}}_{\text{a distribution}} \longrightarrow \underbrace{\{1,\dots,152064\}}_{\text{one integer}} \longrightarrow \underbrace{\mathbb{R}^{3584}}_{e(x_{t+1})}.$$

The middle arrow is the problem. A vector of 3584 real numbers is collapsed into a single integer chosen from 152,064 possibilities, and then re-expanded. Whatever the state was holding that was not expressible as "the identity of one token" is gone, permanently, and no later step can get it back.

Example 7 (the collapse, in miniature)**.** Take a toy model with $d = 3$ and a vocabulary of 4 tokens $V = \{A, B, C, D\}$. Suppose at some step the last hidden state produces logits

$$\ell = (2.0,\ 1.9,\ 0.1,\ 0.0) \quad\Longrightarrow\quad p \approx (0.44,\ 0.40,\ 0.07,\ 0.06).$$

The state is genuinely undecided between A and B: it holds *both* hypotheses, with almost equal weight. Greedy decoding emits A, and the next step's input becomes exactly $e(A)$. The distinction between "I am sure it is A" (say $p = (0.99, 0.003, \dots)$) and "it is probably A but might well be B" has been erased: both produce the identical next input vector $e(A)$.

That example is the entire motivation for what follows. The next Part turns it into a theorem and puts a number on it.

Part III

The decode bottleneck, made precise

Part II ended with an example suggesting that writing reasoning out as text throws information away. This Part proves it, bounds it, and — as importantly — says exactly which architectures the bound does and does not apply to. Getting that scoping right matters: the naive version of this argument is wrong, and it is worth seeing why.

10 Three ways a reasoning step can hand off to the next

(a) Separate-call multi-agent (what CHESS, DIN-SQL, DAIL-SQL actually do). Stage k finishes. Its entire internal state is discarded. Stage $k+1$ is a *new call* with a *fresh context* containing stage k's written output as text.

(b) Single-model chain-of-thought. One continuous forward pass. The internal state of every earlier position is kept in the KV cache and remains attendable. But the *input* at each new position is $e(x_t)$: one of $|V|$ fixed vectors.

(c) Latent (ours). One continuous forward pass. The KV cache is kept *and* the input at each new position is an arbitrary vector in $\mathbb{R}^d$, not a row of W_{in}.

The information-theoretic theorem below is about (a) versus (c). The separate argument in Section 14 is about (b) versus (c). Conflating them produces a claim that is easy to disprove, so we keep them apart.

11 Background: entropy and mutual information

Definition 8 (Entropy)**.** For a random variable X taking values x with probabilities $p(x)$,

$$H(X) = -\sum_x p(x) \log_2 p(x)$$

measured in bits. It is the average number of yes/no questions needed to pin X down. A fair coin has $H = 1$; a constant has $H = 0$.

Definition 9 (Mutual information)**.**

$$I(X;Y) = H(X) - H(X \mid Y),$$

the reduction in uncertainty about X from learning Y. It is 0 when they are independent and equals $H(X)$ when Y determines X. It is symmetric and never negative.

Definition 10 (Markov chain)**.** $X \to Y \to Z$ is a Markov chain if Z depends on X only through Y: formally, X and Z are conditionally independent given Y, so $I(X; Z \mid Y) = 0$.

12 The setup, matching what the code and the baselines actually do

Let $H \in \mathbb{R}^d$ be the internal state that stage k has arrived at: everything it figured out. In architecture (a), stage k must convert H into a token sequence

$$T = \text{decode}(H), \qquad |T| = L \text{ tokens},$$

and stage $k+1$ reconstructs its own understanding by reading that text,

$$H' = \text{encode}(T).$$

Stage $k+1$ has no access whatsoever to the original H — the process that produced it has exited and its memory is freed.

> This is not a modelling approximation. It is a literal description of what a separate-call multi-agent pipeline does. That is precisely why the theorem below applies to it.

13 Theorem: text handoff cannot beat its own token count

Theorem 11 (Data Processing Inequality)**.** *If $H \to T \to H'$ is a Markov chain, then*

$$I(H;T) \;\geq\; I(H;H').$$

Proof. Apply the chain rule for mutual information to the three-variable quantity $I(H;\, T, H')$ in two different orders. Splitting off T first:

$$I(H;T,H') = I(H;T) + I(H;H' \mid T).$$

Splitting off H' first:

$$I(H;T,H') = I(H;H') + I(H;T \mid H').$$

Both equal the same quantity, so they equal each other:

$$I(H;T) + I(H;H' \mid T) = I(H;H') + I(H;T \mid H').$$

By the Markov property, $I(H;H' \mid T) = 0$. Hence

$$I(H;T) = I(H;H') + I(H;T \mid H').$$

Every mutual information, including a conditional one, is non-negative (it is an expectation of a Kullback–Leibler divergence, which is ≥ 0). Therefore $I(H;T \mid H') \geq 0$ and $I(H;T) \geq I(H;H')$. □

Lemma 12 (The token channel has a hard ceiling)**.** *If T is a sequence of at most L tokens from a vocabulary V, then*

$$I(H;T) \;\leq\; H(T) \;\leq\; L \log_2 |V| \text{ bits.}$$

Proof. First, $I(H;T) = H(T) - H(T \mid H) \leq H(T)$, because $H(T \mid H) \geq 0$: you cannot share more information with a variable than that variable contains. Second, there are at most $|V|^L$ distinct sequences of length $\leq L$ (there are $|V|$ choices at each of L positions). The entropy of a variable with N possible values is at most $\log_2 N$, attained only when all are equally likely. Hence $H(T) \leq \log_2(|V|^L) = L \log_2 |V|$. □

Corollary 13 (The handoff bound)**.** *In a separate-call multi-agent pipeline, the information the successor stage can possibly recover about its predecessor's internal state is at most*

$$I(H;H') \;\leq\; L \log_2 |V| \text{ bits,}$$

no matter how the text is written, how clever the prompt is, or how large the successor model is.

Proof. Chain Theorem 11 with Lemma 12. □

13.1 Putting our numbers in

Our model has $|V| = 152{,}064$, so

$$\log_2 |V| = \log_2 152064 \approx 17.214 \text{ bits per token.}$$

Example 14 (one role handoff)**.** A rendered teacher role turn in our corpus — a schema-linking block, say — runs to roughly $L = 200$ tokens. The ceiling is

$$200 \times 17.214 \approx 3{,}443 \text{ bits.}$$

Example 15 (the whole transcript)**.** An entire verbalised debate in our traces averages about $L = 900$ generated tokens, giving

$$900 \times 17.214 \approx 15{,}493 \text{ bits}$$

for the complete multi-stage transcript.

Now the other side. In architecture (c) the successor role does not read text. It continues the same forward pass and attends directly to the predecessor's own KV entries. For our model the per-position key/value state is

$$2 \text{ (keys and values)} \times \underbrace{28}_{\text{layers}} \times \underbrace{4}_{\text{KV heads}} \times \underbrace{128}_{\text{head dim}} = 28{,}672 \text{ numbers,}$$

each stored in `bfloat16`, hence at most 16 bits of entropy each:

$$28{,}672 \times 16 = 458{,}752 \text{ bits per position.}$$

The CHESS-step configuration runs 6 roles $\times$ 2 latent steps = 12 latent positions, so the state the final decode attends to is bounded by

$$12 \times 458{,}752 = 5{,}505{,}024 \approx 5.5 \times 10^6 \text{ bits}$$

of latent working memory — and, critically, *it is not re-encoded.* The successor sees the predecessor's actual tensors, not a reconstruction from prose.

> 5.5×10^6 bits of directly attendable state, versus a 1.5×10^4-bit ceiling on everything a separate-call pipeline can pass between two stages: a factor of roughly 355.

13.2 What this proves, and — importantly — what it does not

Remark 16 (the comparison is between a bound and an exact value, in the right direction)**.** Corollary 13 is an *upper* bound on the text path. The 5.5×10^6 figure is an upper bound on the latent path. Comparing two upper bounds proves nothing in general. What rescues the argument here is that the latent path's transfer is *exact*: no re-encoding step exists, so $I(H; H'_{\text{latent}}) = H(H)$ identically, whereas the text path is capped at $L \log_2 |V|$ regardless of what H contained. The claim "the text handoff is strictly lossy whenever $H(H) > L \log_2 |V|$" is therefore sound; the claim "the latent path actually carries 5.5 million bits of useful reasoning" is not proven and is not claimed.

Remark 17 (the aligned embedding is itself a narrower channel)**.** Section 16 introduces align(h), the map that turns a hidden state back into something usable as an *input* embedding. In its default `soft_embed` form this restricts to the top $k = 32$ logits and returns a convex combination of 32 embedding rows. That is a deterministic function of h, so by the same DPI it can only lose information — it carries roughly the identity of 32 tokens plus 31 free probabilities, on the order of 10^3 bits, not 10^5.

This is not a contradiction: align(h) is the input at the *next* position, and it is a genuinely narrow pipe. The wide channel is the KV cache, which the successor attends to in parallel and which nothing compresses. Both statements are true simultaneously, and stating only the flattering one would be dishonest. `align="identity"` makes the narrow pipe lossless too, and is implemented; it is not the default because a raw hidden state is out of distribution as an input embedding and empirically produced degenerate output.

Remark 18 (capacity is not use)**.** Nothing above shows the model *fills* the latent channel with independent, useful reasoning rather than 12 near-copies of the same state. That question belongs to training, where the guarantees are far weaker (Part VIII, Part X).

14 The second, different argument: superposition and search

The DPI argument above compares (a) with (c). It says nothing about (b), a single-model chain-of-thought that keeps its KV cache. For that comparison there is a different mechanism, and it is the one Coconut's own experiments identify.

Return to the toy example of Part II: a state with $p \approx (0.44, 0.40, 0.07, 0.06)$ over $\{A, B, C, D\}$. In architecture (b) the next input must be one of the four fixed vectors $e(A), \ldots, e(D)$. The model must *commit*. In architecture (c) the next input is

$$\text{align}(h) \approx 0.44\, e(A) + 0.40\, e(B) + 0.07\, e(C) + 0.06\, e(D),$$

a point strictly between them that no token corresponds to. Both hypotheses survive into the next step.

> Committing to a token is a *greedy* search step. Passing a mixture keeps a *frontier*. The first is depth-first with no backtracking; the second is closer to breadth-first.

Example 19 (why postponing helps, concretely)**.** Suppose two candidate schema linkings, A and B, are near-equally plausible given the question alone, but only one survives once the query is actually written. A token-committed chain picks A at the linking step, and every later step conditions on A; if A was wrong, nothing downstream can recover, because the evidence that would have discriminated arrives *after* the commitment. A latent chain carries $0.44A + 0.40B$ forward, and the later step — which now has the evidence — can still resolve toward B.

Coconut measures exactly this on a planning benchmark: probing the latent state shows the model assigning high value to a node it did *not* rank first at the previous step, i.e. behaving non-greedily, and the gap between the top-1 and top-3 cumulative values is wide at the first latent step and narrows at the second — broad exploration first, then commitment. This is the concrete content of the phrase "latent reasoning explores a broader search space".

Remark 20 (status of this argument)**.** This is a mechanism plus published empirical support on a different task, not a theorem about our system. We have not probed our own latent states for superposition. It is stated here as the motivation it is.

Part IV
The continuous thought

We now build the mechanism itself, in four steps, each using only the previous one.

15 Step 1: latent mode

Definition 21 (Language mode and latent mode)**.** A model in **language mode** at position t does

$$h_t \;\to\; \text{softmax}(W_{\text{out}} h_t) \;\to\; x_{t+1} \;\to\; e(x_{t+1}),$$

i.e. equation (2.3). A model in **latent mode** skips the middle entirely:

$$\boxed{\text{next input embedding} \;=\; \text{align}(h_t)} \tag{4.1}$$

No token is sampled, no token is emitted, and the position contributes nothing to the output text.

Coconut states the same thing as a modification of the input matrix. If latent mode runs between positions i and j, then for $i < t < j$,

$$E_t = \big[e(x_1), \ldots, e(x_i),\; h_i,\; h_{i+1}, \ldots, h_{t-1}\big],$$

and once latent mode ends the input reverts to ordinary embeddings:

$$E_t = \big[e(x_1), \ldots, e(x_i),\; h_i, \ldots, h_{j-1},\; e(x_j), \ldots, e(x_t)\big].$$

The vector h_t produced in latent mode is called a **continuous thought**. The quantity $p(x_{t+1} \mid x_{\le t})$ is simply not defined at those positions: nothing is being predicted there. (One *can* still compute $\text{softmax}(W_{\text{out}} h_t)$ for diagnostic purposes, which is how Coconut probes what a latent state is "thinking"; nothing in training uses it.)

> A latent step costs **one forward pass** and emits **zero tokens**. That is the entire efficiency claim, and it is exact, not statistical.

16 Step 2: the alignment map

Equation (4.1) has an `align` in it. Why not feed h_t back directly?

Because h_t and $e(x)$ live in the same space $\mathbb{R}^d$ but are not the same *kind* of object. $e(x)$ is a row of W_{in}: it comes from a set of 152,064 specific vectors that the first transformer layer has seen during all of pretraining. h_t is an output of the last layer after final normalisation. Feeding one where the other is expected is an *out-of-distribution* input. The project measured this: with the identity map the model produced degenerate token soup at evaluation.

Three constructions are implemented in `latentsql/distill/latent.py::build_aligner`.

16.1 Construction 1: identity (Coconut's own choice)

$$\text{align}(h) = h.$$

Kept for ablation. It is the only construction that is information-lossless in the narrow-pipe sense of Section 13.2.

16.2 Construction 2: linear

The natural way to get from hidden space back to input space is to go through the vocabulary and come back:

$$h \xrightarrow{W_{\text{out}}} \text{logits} \in \mathbb{R}^{|V|} \xrightarrow{W_{\text{in}}} \mathbb{R}^d.$$

Composing the two matrices once, up front, gives a single $d \times d$ matrix:

$$W_a = W_{\text{out}}^\top W_{\text{in}} \in \mathbb{R}^{d \times d}, \qquad \text{align}(h) = h\, W_a. \tag{4.2}$$

Remark 22 (a scale problem, and the fix)**.** W_a sums over all 152,064 vocabulary entries, so hW_a has a norm orders of magnitude away from that of a real token embedding — feeding it in directly would present the model with either a near-zero or an exploding input. The code therefore rescales once at construction time:

$$W_a \;\leftarrow\; W_a \cdot \frac{\overline{\|W_{\text{in}}[v,:]\|}}{\overline{\|zW_a\|}}, \qquad z \sim \mathcal{N}(0, I),$$

i.e. it measures the average norm of a real input embedding, measures the average norm W_a produces on random probes, and scales so the two agree.

Remark 23 (relation to the published form)**.** LatentMAS writes the alignment as $W_a \approx W_{\text{out}}^{+} W_{\text{in}}$ with a pseudo-inverse. With the shape convention logits $= hW_{\text{out}}^\top$ used here, the composition that actually maps hidden $\to$ vocabulary $\to$ embedding is $W_{\text{out}}^\top W_{\text{in}}$, which is what is implemented. The two agree up to a logit temperature.

16.3 Construction 3: soft embedding (the default)

Rather than a first-order linear surrogate, take the token distribution the hidden state actually encodes, restricted to its top k entries, and return the corresponding convex combination of embedding rows:

$$\ell = W_{\text{out}} h, \qquad \mathcal{K} = \text{top-}k(\ell), \qquad \pi_v = \frac{\exp(\ell_v/\tau)}{\sum_{u \in \mathcal{K}} \exp(\ell_u/\tau)} \;\; (v \in \mathcal{K}),$$

$$\boxed{\text{align}_{\text{soft}}(h) \;=\; \sum_{v \in \mathcal{K}} \pi_v \, W_{\text{in}}[v,:]} \tag{4.3}$$

with $k = 32$ and $\tau = 1$ by default.

> Equation (4.3) is precisely the "keep the frontier" object of Section 14, written down. It is a weighted blend of the 32 tokens the model currently considers plausible — a point in embedding space that no single token occupies — and it is guaranteed in distribution, because it is a convex combination of real rows of W_{in}.

Example 24 (the toy model again)**.** With $\ell = (2.0, 1.9, 0.1, 0.0)$ over $\{A, B, C, D\}$ and $k = 2$:

$$\pi_A = \frac{e^{2.0}}{e^{2.0} + e^{1.9}} = \frac{7.389}{7.389 + 6.686} = 0.5250, \qquad \pi_B = 0.4750,$$

so

$$\text{align}_{\text{soft}}(h) = 0.5250\, e(A) + 0.4750\, e(B).$$

Greedy decoding would have produced $e(A)$ and discarded B entirely.

A definition needed before the lemma: convex combination and convex hull

The lemma below uses two terms that are not assumed known. Both are simple, and both are worth building from a picture rather than a formula first.

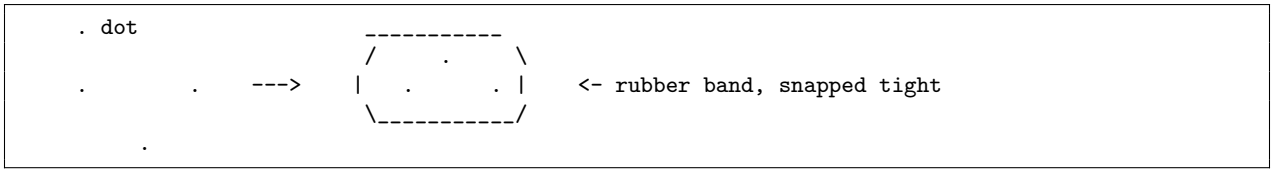
Definition 25 (Convex combination)**.** A **convex combination** of points $p_1, \ldots, p_K$ is any weighted average

$$w_1 p_1 + w_2 p_2 + \cdots + w_K p_K, \qquad w_i \geq 0, \qquad \sum_{i=1}^{K} w_i = 1.$$

The two conditions on the weights matter individually. Take two points A and B on a page. If the weights must be non-negative and sum to 1, the only points you can reach by mixing A and B are the ones *on the straight line segment between them* — $0.5A + 0.5B$ is the midpoint, $0.9A + 0.1B$ is a point close to A, and so on. Drop the "non-negative" requirement and you could take a weight of -0.5 on B, which points *away* from B and shoots straight past A out the far side — off the segment entirely. Drop the "sum to 1" requirement and $2A + 2B$ scales the whole thing outward, again leaving the segment. Both restrictions together are exactly what keeps a weighted mix trapped between the points, never past them.

Definition 26 (Convex hull)**.** The **convex hull** of a set of points is every point reachable as a convex combination of some (or all) of them.

Picture a handful of dots scattered on a page, and a rubber band stretched around the outermost ones and let snap tight, as drawn below. Every point trapped inside the band — the dots themselves and everything between them — is the convex hull. A dot already surrounded by others adds nothing new; the band only bends around the outermost ones. And this is not just an analogy: it is exactly what Definition 25 says algebraically — non-negative weights summing to 1 is precisely what keeps a mix from escaping the band.



Now scale the picture up. Instead of a handful of dots on a 2-number page, our vocabulary gives 152,064 "dots" (one embedding row per token), each described by 3584 numbers instead of 2. The idea does not change — there is still an (imaginary, 3584-dimensional) rubber band stretched around all 152,064 of them, and the region it traps is the convex hull of $\{W_{\text{in}}[v,:] : v \in V\}$: precisely *the neighbourhood every real token embedding lives in*, plus every point reachable by non-negative-weighted mixes of them.

Setup, so the lemmas below are about h and W_{in} explicitly. Fix a hidden state $h \in \mathbb{R}^d$. It determines a logit vector $\ell = W_{\text{out}} h \in \mathbb{R}^{|V|}$, a top-$k$ index set $\mathcal{K} = \mathcal{K}(h) = \text{top-}k(\ell)$, and hence a *finite, h-dependent* candidate set of embedding rows

$$S_h \;:=\; \{\, W_{\text{in}}[v,:] \;:\; v \in \mathcal{K}(h) \,\} \subset \mathbb{R}^d, \qquad |S_h| = k = 32.$$

By (4.3), $\text{align}_{\text{soft}}(h) = \sum_{v \in \mathcal{K}} \pi_v \, W_{\text{in}}[v,:]$ is a convex combination of exactly the 32 points of S_h — *not* of all 152,064 rows of W_{in}. That distinction is what the weak lemma below misses and the two lemmas after it repair.

Lemma 27 (weak containment)**.** $\mathrm{align}_{\mathrm{soft}}(h) \in \mathrm{conv}\big(\{W_{\mathrm{in}}[v,:] : v \in V\}\big)$.

Proof. The π_v are non-negative and sum to 1 (softmax), so (4.3) is a convex combination of the points of S_h; and $S_h \subseteq \{W_{\mathrm{in}}[v,:] : v \in V\}$, so $\mathrm{conv}(S_h) \subseteq \mathrm{conv}\big(\{W_{\mathrm{in}}[v,:] : v \in V\}\big)$ by monotonicity of the convex-hull operator under set inclusion. □

This says only "inside the rubber band around all 152,064 tokens". With $|V| = 152{,}064 \gg d = 3584$, that hull is generically full-dimensional — it fills essentially the entire 3584-number space — so Lemma 27 alone rules out almost nothing. The next two lemmas use the actual value of h, not just its existence, to say something much smaller.

Lemma 28 (sharp containment: a 31-simplex, not the whole hull)**.** $\mathrm{align}_{\mathrm{soft}}(h) \in \mathrm{conv}(S_h)$*, and generically (when the* 32 *points of* S_h *are affinely independent, the typical case since* $d = 3584 \gg k - 1 = 31$*) this set is a* 31*-dimensional simplex — the convex hull of exactly* 32 *vertices, an object of dimension* 31 *sitting inside a* 3584*-dimensional ambient space.*

Proof. Let $E_h \in \mathbb{R}^{d\times k}$ be the matrix whose i-th column is the i-th row of S_h, and let $\Delta^{k-1} = \{w \in \mathbb{R}^k : w_i \geq 0, \sum_i w_i = 1\}$ be the standard simplex. The map $\phi_h : \Delta^{k-1} \to \mathbb{R}^d$, $\phi_h(w) = E_h w$, is linear and, by definition of a convex hull, $\phi_h(\Delta^{k-1}) = \mathrm{conv}(S_h)$. The softmax weight vector $\pi = (\pi_v)_{v\in\mathcal{K}}$ satisfies $\pi \in \Delta^{k-1}$ (non-negative, sums to 1), and $\mathrm{align}_{\mathrm{soft}}(h) = \phi_h(\pi)$ by (4.3). So $\mathrm{align}_{\mathrm{soft}}(h) \in \phi_h(\Delta^{k-1}) = \mathrm{conv}(S_h)$. Δ^{k-1} has affine dimension $k - 1 = 31$; an injective linear map preserves affine dimension, and 32 points in general position in $\mathbb{R}^{3584}$ are affinely independent (a codimension-0, i.e. generic, condition), so ϕ_h is injective and $\mathrm{conv}(S_h)$ has dimension 31. □

Lemma 29 (genuine mixing, not a corner)**.** *For any finite temperature* $\tau > 0$*, every* $\pi_v > 0$ *strictly, so* $\mathrm{align}_{\mathrm{soft}}(h)$ *lies in the* relative interior *of* $\mathrm{conv}(S_h)$ *— never on a face, edge, or vertex of the simplex.*

Proof. $\pi_v = \exp(\ell_v/\tau)/\sum_{u\in\mathcal{K}} \exp(\ell_u/\tau)$, and $\exp(x) > 0$ for every finite real x, so every $\pi_v > 0$. A convex combination with a strictly positive coefficient on *every* vertex of a simplex is, by definition, in that simplex's relative interior (a point on a proper face has coefficient exactly 0 on at least one vertex). □

Lemma 30 (quantitative localisation: how far from the model's own top guess)**.** *Let* $v^\star = \arg\max_{v\in\mathcal{K}} \pi_v$ *be the single most likely token. Then*

$$\big\|\, \mathrm{align}_{\mathrm{soft}}(h) - W_{\mathrm{in}}[v^\star,:] \,\big\| \;\leq\; \max_{v\in\mathcal{K}} \big\|\, W_{\mathrm{in}}[v,:] - W_{\mathrm{in}}[v^\star,:] \,\big\|.$$

Proof. Since $\sum_{v\in\mathcal{K}} \pi_v = 1$, subtract $W_{\mathrm{in}}[v^\star,:]$ from (4.3) using $\pi_v W_{\mathrm{in}}[v^\star,:]$ summed over v equal to $W_{\mathrm{in}}[v^\star,:]$:

$$\mathrm{align}_{\mathrm{soft}}(h) - W_{\mathrm{in}}[v^\star,:] = \sum_{v\in\mathcal{K}} \pi_v \big(W_{\mathrm{in}}[v,:] - W_{\mathrm{in}}[v^\star,:]\big).$$

Apply the triangle inequality, then $\pi_v \geq 0$ and $\sum_v \pi_v = 1$:

$$\big\|\mathrm{align}_{\mathrm{soft}}(h) - W_{\mathrm{in}}[v^\star,:]\big\| \leq \sum_{v\in\mathcal{K}} \pi_v \big\|W_{\mathrm{in}}[v,:] - W_{\mathrm{in}}[v^\star,:]\big\| \leq \Big(\max_{v\in\mathcal{K}} \big\|W_{\mathrm{in}}[v,:] - W_{\mathrm{in}}[v^\star,:]\big\|\Big) \sum_{v\in\mathcal{K}} \pi_v,$$

and the last sum is 1. □

What the four lemmas jointly say. $\text{align}_{\text{soft}}(h)$ is not merely "somewhere in the 152,064-token hull" (Lemma 27, weak). It is a specific, strictly-interior point (Lemma 29) of a 31-dimensional simplex spanned by only the 32 embeddings h *itself* finds most plausible (Lemma 28), at a distance from the model's own greedy top-1 choice that never exceeds the local spread of those same 32 candidates (Lemma 30). `identity` mode (Construction 1) has none of these guarantees: h is the *last* layer's output, unrelated by construction to the 152,064 rows of W_{in} the *first* layer was trained on, so no lemma above applies to it at all — consistent with the degenerate token soup measured at evaluation.

17 Step 3: latent working memory (the KV cache)

A transformer is causal, so when position $t+1$ is computed, the keys and values of positions $1..t$ are unchanged. They are cached rather than recomputed. That cache is

$$\mathcal{C}_t = \left\{(K^{(\ell)}_{1:t},\, V^{(\ell)}_{1:t})\right\}_{\ell=1}^{28},$$

one key/value pair per layer per position.

In our chain we carry $\mathcal{C}$ across role boundaries. That is exactly the statement that role $k+1$ attends to role k's actual representations rather than to a re-encoding of role k's prose. It is the wide channel of Section 13.2.

`share_working_memory:` `true` keeps $\mathcal{C}$ across roles.
`share_working_memory:` `false` resets it at each role boundary — the ablation that simulates a separate-call pipeline.

17.1 A subtlety the code had to solve: cache mutation under checkpointing

This subsection can be skipped on a first reading, but it is the kind of detail that determines whether the gradients are correct.

To fit training in memory, the m latent steps of a role are wrapped in `torch.utils.checkpoint`: their intermediate activations are not stored, they are *recomputed* during the backward pass. Non-reentrant checkpointing therefore calls the wrapped function **twice** — once now, once during backward.

The problem: the KV cache is mutated *in place*. Each `model(..., use_cache=True)` appends to it. A blind second call would append the same m steps again, so the cache length would be wrong for every role that already ran on top of it.

The fix, at `latent.py:220`, is one line at the top of the checkpointed function:

```
past_key_values.crop(start_len)   # no-op first call; undoes 2nd call's own append
```

`crop` is a plain tensor slice, hence differentiable, and a true no-op when the cache is already at that length. So:

- **First (real) call:** the cache is already at `start_len`; cropping does nothing; m steps are appended.
- **Second (backward-time) call:** forward has completed, so the cache is at `start_len`+m; cropping removes exactly this role's own extension, and the m steps are then redone identically.

The property being restored is **idempotence**: running the block once and running it twice must leave the same state. That is what non-reentrant checkpointing requires, and it is verified in `tests/test_core.py` by comparing LoRA gradients computed with and without checkpointing — they agree to float32 tolerance.

18 Step 4: the cost accounting

Let R be the number of latent roles and m the number of latent steps per role. Then per question:

	Explicit pipeline	**Latent (ours)**
Model calls	one per stage ($\approx$ 6–12)	one
Forward passes	one per generated token, per stage	Rm + one per SQL token
Tokens generated	≈ 1339 (measured)	≈ 61 (measured)
Tokens read	each stage re-reads the schema	schema read once
Cross-stage channel	text, $\leq L \log_2 \lvert V \rvert$ bits	KV cache, no re-encoding

With $R = 6$ and $m = 2$ this is 12 latent forward passes — fewer than the cost of generating 12 tokens, and it replaces roughly 1280 of them.

Remark 31 (what the token counts in this document mean)**.** `Avg tokens` is prompt_tokens + completion_tokens. Latent steps are *not* counted as completion tokens, because they are not tokens; they are tracked separately as `extra.latent_steps`. Anywhere a comparison could be read as hiding the latent cost, both numbers are given.

19 Is this beam search? Is it tree search? A precise comparison

A latent step (4.3) looks at the top-k next-token candidates and reasons about several of them at once. That invites a natural question: is this secretly beam search, or Monte Carlo Tree Search (MCTS), under a different name? **No, to both** — and the reason is worth stating precisely, since "explore several options" and "search over several options" sound alike but are mechanically opposite.

19.1 Before any formal definition: the exact same numbers, two ways

Reuse the worked example from Construction 3 (§16): a hidden state h whose top-2 candidates are tokens A and B, with softmax weights $\pi_A = 0.525$, $\pi_B = 0.475$ (worked out there in full). Trace one step forward, the beam-search way and our way, side by side.

Beam search, $K = 2$, starting from this exact point		
Step	What happens	Result
1	Keep the top 2 tokens as **two separate, complete copies** of the sequence so far — one ending in A, one ending in B.	Two sequences now exist, each with its own KV cache.
2	Run the model **twice**: once continuing from "$\ldots A$", once from "$\ldots B$". Say this gives $(0.6, 0.4)$ over $\{X, Y\}$ after A, and $(0.3, 0.7)$ over $\{X, Y\}$ after B.	Four candidate two-token sequences exist: AX, AY, BX, BY, each with its own cumulative score.
3	Score all four, keep only the best 2 — say BY and AX win.	AY and BX are **thrown away for good**. If the best possible completion started with BX, it is gone, even though B itself was kept.

Our latent step, at the exact same point		
Step	What happens	Result
1	Do not pick A or B. Blend them into one vector: $\text{align}_{\text{soft}}(h) = 0.525\, e(A) + 0.475\, e(B)$.	**One** sequence still exists, one KV cache.
2	Run the model **once**, on that one blended vector.	One next-state distribution comes out. There was never a second, separate "B-branch" sitting around to compare against or discard.
3	(there is no step 3 — nothing was kept apart to choose between)	Nothing is ever thrown away, because nothing was ever split off in the first place.

The whole difference, in one line. Beam search keeps A's branch and B's branch alive as two separate things, extends both, and only *later* throws one away. The latent step never lets A and B become separate things to begin with — so there is nothing later to compare, nothing to discard, and no second forward pass was ever spent computing the branch that beam search would have thrown away anyway.

This is exactly what the formal claim below (Proposition 34) says: at no point in the second table does a second copy of anything exist.

Definition 32 (Beam search)**.** Maintain K separate, explicit token sequences ("beams"). At every step, extend each of the K beams by its own candidate next tokens, score every resulting continuation by cumulative log-probability, and keep only the top K continuations for the next step. At the end, the single highest-scoring completed sequence is returned.

Definition 33 (Monte Carlo Tree Search)**.** Build an explicit tree whose nodes are partial sequences. Repeatedly: descend the tree toward promising nodes (*selection*), extend a leaf with a new candidate (*expansion*), estimate that candidate's value by rolling forward to a full sequence and scoring it (*simulation*), and propagate that score back up the tree to update every ancestor's estimate (*backpropagation*). After many such iterations, commit to the highest-value child at the root.

Proposition 34 (the latent step is neither)**.** *At every position in the sequence (4.3), (5.1), there is exactly* ***one*** *input embedding, one hidden state, one KV cache. No step in* `_latent_steps` *or* `align_soft` *creates a second copy of the sequence, scores two candidates against each other, or discards one in favour of another.*

Proof. Immediate from the definition of $\text{align}_{\text{soft}}$ (4.3): it is a single function $\mathbb{R}^d \to \mathbb{R}^d$, returning one vector $\sum_{v \in \mathcal{K}} \pi_v W_{\text{in}}[v, :]$. There is no data structure in `_latent_steps` (Part IV) holding more than one hypothesis: `pieces` is a single growing list, `past_key_values` a single cache. Beam search and MCTS both require, by their own definitions above, more than one candidate to exist *simultaneously* so that they can be compared; here only one ever exists. □

	Beam search / MCTS	**GRPO** (Part X)	**Latent step (ours)**
When it runs	inference time	training time	inference *and* training
How many sequences exist at once	K, kept explicit and compared	G full rollouts, compared *after* decoding	1 — always
What changes as a result	which sequence is *output*	the model's *weights*	nothing is compared; the state itself is a blend
Extra inference cost	$K\times$ or more forward passes	none (training-only)	**zero** — same cost as one token

The one-sentence answer. Beam search and MCTS get their exploration by keeping K things separate and comparing them; the latent step gets a *different* kind of exploration — Section 14's superposition — by keeping exactly 1 thing and making that one thing a blend. It is cheap for exactly the reason it cannot literally be a search: there is nothing to compare, because nothing was ever kept apart.

Remark 35 (why not use real search here)**.** MCTS-style search at inference would reintroduce precisely the cost this project exists to remove: CHESS's own multi-candidate generation (Part I) already pays several LLM calls per question for exactly this reason. A value estimate for a *partial* SQL query or a partial latent state is also not cheaply available the way it is in a board game (no partial query can be executed against the database to check if it is "winning"), which is the other standard prerequisite MCTS needs and this setting does not supply for free.

Remark 36 (and the final SQL decode is not a search either)**.** The one place a real, discrete choice *is* made is decoding the SQL tokens after `<|consensus|>` (Part V). Evaluation runs this at `temperature=0` (`configs/latent_sft_1000_7b_bf16.yaml`), so `LatentReasoner.generate` takes a plain greedy argmax at every position (`latent.py`, the decode loop after the consensus tag) — one path, no beam, no rescoring. GRPO's `group_size` rollouts (Part X) do sample G different SQL completions per prompt, but only to *compute a training reward*; by Proposition 68, every one of those G completions is decoded from the identical shared latent prefix, so even there no search over *latent* states occurs — only the decoded text differs.

Part V
From a multi-agent pipeline to a chain of latent slots

Part IV built one latent step. This Part says what the latent steps are *for*.

20 The role chain

We take CHESS's decomposition and write it as an ordered list of six roles plus a terminator:

$$\rho = \big(\underbrace{r_1,\ r_2,\ r_3,\ r_4,\ r_5,\ r_6}_{\text{latent-trainable}},\ \mathsf{consensus}\big)$$

with

k	role r_k	what this stage does
1	`information_retriever`	find literal values from the question that occur in the database
2	`schema_linker`	select the relevant tables and columns
3	`query_classifier`	label the query easy / non-nested / nested
4	`candidate_generator`	write a first candidate query
5	`query_fixer`	run it, read the error, correct it
6	`unit_tester`	check the corrected query answers the question
–	`consensus`	*not* a reasoning stage; the marker after which SQL is emitted

consensus is a punctuation mark, not an agent. It is the token that says "deliberation is over; the next thing you produce is the query". Almost everything in Part IX turns on whether the model still believes that.

21 The tag vocabulary, and why it is load-bearing

Each role is announced by a literal string:

$$\mathsf{tag}(r) = \texttt{<|agent}\ r\ \texttt{|>}, \qquad \mathsf{tag}(\mathsf{consensus}) = \texttt{<|consensus|>}.$$

These are not decoration. IMAD is explicit on the point: without structure tags, "the distinction between agents becomes difficult to make", and IMAD's own activation-space analysis (Part VI) finds that models trained *without* the tags show measurably less separable agent subspaces.

The mechanism is straightforward once stated. A latent step's content is entirely determined by the state it starts from. The tag is what sets that state: it is the anchor that tells the model *which* of the six jobs the next m vectors are supposed to be doing. Remove the tag and all six latent blocks start from indistinguishable states and there is no reason for them to specialise.

22 The sequence, written out

Let c be the rendered chat prompt (system message + user message containing the schema, the question, any evidence, and retrieved value hints), and let $y^\star$ be the gold SQL target. Write $\bullet$ for one latent position.

The **fully latent** sequence, which is what evaluation runs, is

$$\underbrace{c}_{\text{prompt}} \,\|\, \mathsf{tag}(r_1)\, \underbrace{\bullet\bullet}_{m=2} \,\|\, \mathsf{tag}(r_2)\; \bullet\bullet \,\|\, \cdots \,\|\, \mathsf{tag}(r_6)\; \bullet\bullet \,\|\, \texttt{<|consensus|>} \,\|\, y^\star. \tag{5.1}$$

Only the last block is decoded. The six tags are fed as text but not generated (they are teacher-forced in training and inserted by the runner at evaluation); the twelve $\bullet$ positions emit nothing at all.

Example 37 (what a real prediction looks like)**.** Input: a Spider question over a museum database. Output — the *entire* generated text, 61 tokens:

````
```sql
SELECT T2.name FROM visit AS T1 JOIN visitor AS T2
  ON T1.visitor_id = T2.visitor_id
GROUP BY T1.visitor_id HAVING COUNT(*) > 1;
```
````

No schema-linking prose, no candidate discussion, no unit tests. All six of those stages happened; none of them were written down.

23 The curriculum: stage k

Asking a model that has only ever written prose to jump straight to (5.1) does not work; the project measured that too. Coconut's answer, inherited from iCoT (Deng et al.), is a **curriculum**: hide the roles one at a time.

Definition 38 (Stage k)**.** At stage $k \in \{0, 1, \ldots, 6\}$, the first k roles are latent and the rest are dropped. The sequence is

$$c \,\|\, \mathsf{tag}(r_1)\; \bullet\bullet \,\|\, \cdots \,\|\, \mathsf{tag}(r_k)\; \bullet\bullet \,\|\, \texttt{<|consensus|>} \,\|\, y^\star. \tag{5.2}$$

> **Read Definition 38 twice.** The roles after the k-th are not written out as text. They are **dropped entirely**. Stage k trains "after k latent roles, emit the SQL". This is a deliberate choice — `chain-format: chess_steps`, called "SQL-direct" — and it is not what Coconut does. Coconut keeps the remaining reasoning steps as text. The consequences of this difference are the entire subject of Part IX.

Example 39 (stage 3 versus stage 4, side by side)**.**

```
stage 3:  prompt | r1 oo | r2 oo | r3 oo |                <|consensus|> | SQL
stage 4:  prompt | r1 oo | r2 oo | r3 oo | r4 oo |        <|consensus|> | SQL
stage 5:  prompt | r1 oo | r2 oo | r3 oo | r4 oo | r5 oo | <|consensus|> | SQL
```

Each stage adds one tag and two latent positions. Nothing else changes: the prompt is the same, the target is the same, and the number of supervised tokens is the same.

23.1 How the curriculum is scheduled in practice

Two mechanisms exist in the code:

1. **In-process ramp.** With `curriculum=True` and no override, the depth is a linear function of the step index:

$$\text{frac} = \frac{\text{step}}{\max(1, n_{\text{steps}} - 1)}, \qquad k = \text{round}(\text{frac} \cdot R).$$

2. **Chain mode (what actually ran).** One training *process* per batch, with the stage passed in explicitly by the shell runner via `--force-n-latent-roles`. This exists because a long-lived process was observed to corrupt CUDA/checkpoint state after an out-of-memory event; a fresh process per batch sidesteps it.

Chain mode has a consequence that matters later: a brand new `AdamW` is constructed in every process. The runner therefore saves and restores the optimiser state (`--optim-state`) so that momentum is continuous across the chain — *except* at stage boundaries, where the runner deliberately deletes it:

```
if [ "$ep_start" -eq 1 ]; then
  rm -f "$OPTIM_STATE"
fi
```

This follows Coconut, which also resets the optimiser at each stage switch. Proposition 56 explains why that reset is more dangerous than it looks.

Part VI
Activation space: why any of this can work

There is a question that should be nagging by now. We are asking one set of weights to behave, in sequence, like six different specialists. Why should a single model be able to hold six distinct "personas" at all, rather than averaging them into one mush?

IMAD answers this question empirically, and the answer is the theoretical foundation this project rests on. This Part states it, and proves the one piece of mathematics it depends on.

24 The residual stream and the linear representation hypothesis

A transformer layer does not overwrite its input; it *adds* to it:

$$h^{(\ell+1)} = h^{(\ell)} + \mathrm{Attn}^{(\ell)}\big(h^{(\ell)}\big) + \mathrm{MLP}^{(\ell)}\big(h^{(\ell)}\big).$$

The running sum $h^{(\ell)}$ is called the **residual stream**. Because every layer contributes additively, the stream behaves like a shared vector workspace that layers read from and write to.

> **Linear representation hypothesis.** Many human-interpretable properties of a model's state — "this text is harmful", "this is code", "this is agent 3 speaking" — are encoded approximately as *directions* in the residual stream: there is a vector v such that the property's presence corresponds to a large value of $v \cdot h$.

This is a hypothesis, not a theorem. Its evidence is that interventions built on it work, which is what the next two sections are about.

25 Difference-in-means, and a proof that it is the right direction

Definition 40 (Difference-in-means)**.** Given activations from two classes at a fixed layer ℓ — positives $h_1^+, \dots, h_N^+$ and negatives $h_1^-, \dots, h_N^-$ — the **difference-in-means** direction is

$$d = \mu_+ - \mu_-, \qquad \mu_\pm = \frac{1}{N}\sum_{i=1}^{N} h_i^\pm, \qquad \hat{d} = \frac{d}{\|d\|}. \tag{6.1}$$

This is what `latentsql/steering/caa.py` computes, and it is IMAD's equation (3):

$$v_i = \frac{1}{|\mathcal{D}|}\sum_{(p,c)\in\mathcal{D}} \Big[h_\ell(p, c_i) - h_\ell(p, c_{\neg i})\Big],$$

where the context p is held *identical* and only the continuation varies: c_i is agent i's real response, $c_{\neg i}$ the average over the other agents' responses in the same slot.

Why the difference of means, out of every direction in a 3584-dimensional space? Because under one clean assumption it is provably optimal.

Assumption 41 (isotropic noise)**.** Model class-$+$ activations as $h = \mu_+ + \varepsilon$ and class-$-$ activations as $h = \mu_- + \varepsilon$, with the noise ε having the same spread in every direction:

$$\mathbb{E}\big[(w \cdot \varepsilon)^2\big] = \sigma^2 \quad \text{for every unit vector } w.$$

Lemma 42 (Cauchy–Schwarz)**.** *For any vectors a, b: $|a \cdot b| \le \|a\|\|b\|$, with equality exactly when a and b are parallel.*

Proof. For any real t, a squared length is non-negative:

$$0 \le \|a - tb\|^2 = \|a\|^2 - 2t(a \cdot b) + t^2\|b\|^2.$$

This is a quadratic in t that is never negative, so its discriminant is ≤ 0:

$$\big(2(a \cdot b)\big)^2 - 4\|a\|^2\|b\|^2 \le 0 \implies (a \cdot b)^2 \le \|a\|^2\|b\|^2.$$

Taking square roots gives the inequality. Equality forces $\|a - tb\|^2 = 0$ for some t, i.e. $a = tb$. □

Theorem 43 (Difference-in-means is the optimal linear probe under Assumption 41)**.** *Among all unit directions w, the one maximising Fisher's separation score*

$$J(w) = \frac{\big(w \cdot \mu_+ - w \cdot \mu_-\big)^2}{\mathrm{Var}(w \cdot h)}$$

is $w^\star = \hat{d} = (\mu_+ - \mu_-)/\|\mu_+ - \mu_-\|$.

Proof. By Assumption 41 the denominator equals σ^2 for *every* unit w; it is therefore a constant of the optimisation and can be dropped. The numerator is $(w \cdot d)^2$ with $d = \mu_+ - \mu_-$. By Lemma 42, for unit w,

$$|w \cdot d| \le \|w\|\,\|d\| = \|d\|,$$

with equality exactly when $w \parallel d$, i.e. $w = d/\|d\|$. Hence

$$\boxed{w^\star = \hat{d}}$$

maximises $(w \cdot d)^2$ and therefore $J(w)$. □

Remark 44 (what breaks without the assumption)**.** Real residual streams are not isotropic: some directions carry systematically more variance, and directions are correlated. Under a general shared covariance Σ the Fisher-optimal direction is

$$w^\star \propto \Sigma^{-1}(\mu_+ - \mu_-),$$

a whitened difference-in-means that down-weights noisy directions. Estimating and inverting a 3584×3584 covariance from a few hundred contrastive examples is ill-posed, which is the practical reason both this project's `caa.py` and the published steering work use the raw difference. This is a real gap between the theorem and the code, and it is stated rather than papered over.

26 Steering: using the direction

Definition 45 (Contrastive activation addition)**.** At inference, hook layer ℓ and add a multiple of the direction to its output:

$$h_\ell \;\longleftarrow\; h_\ell + \alpha\, v_i. \tag{6.2}$$

$\alpha > 0$ amplifies the property, $\alpha < 0$ suppresses it, $\alpha = 0$ is the unmodified model.

This is implemented as a forward hook in `caa.py::steer_layer`, at layer -8 (eight from the end) by default.

27 IMAD's finding, and what it licenses

IMAD trains a model to internalise a three-agent debate where each agent has a deliberately distinct style (chain-of-thought, self-critique, program-of-thought), extracts v_1, v_2, v_3 by (6.1), and then steers by (6.2) while measuring how closely the steered output resembles that agent's real responses.

Result. The internalised model is steerable toward each individual agent *substantially better than the base model is*: ROUGE-L AUC 0.802 $\rightarrow$ 0.903, a 12.7% improvement, with per-agent, per-metric gains from 6.1% to 25.0% and an average of 15.4%. At $\alpha = 0$ the two models are by construction identical, so the entire effect is attributable to internalisation having created structure that steering can find.

The interpretation, and the foundation of this project: internalising a multi-agent system does not blend the agents into an undifferentiated policy. It creates *agent subspaces* — separable directions in activation space, one per role. The agents survive internalisation as geometry.

That is the answer to the question this Part opened with. One model *can* hold six specialists, because the specialists occupy different directions in a 3584-dimensional space and there is plenty of room. It also explains why the role tags matter (Part V): the tag is what selects which subspace the following latent steps operate in.

IMAD tests the claim in the other direction too, which is a stronger test than the positive one: they deliberately instil a *malicious* agent through the same pipeline, then apply *negative* steering ($\alpha < 0$) to that agent's direction. Suppression is more effective after internalisation than on the base model, and costs less general capability — on GSM8K the internalised model stays stable across the full steering range while the base model's accuracy collapses at extreme coefficients. A behaviour that has been given its own direction is a behaviour you can find and remove.

28 What this does and does not establish for LatentSQL

Established	Not established
Internalised multi-agent training produces separable per-agent directions, in IMAD's setting, measured by steering.	That *our* six CHESS-role latent slots occupy separable subspaces. We have the extraction code and have not run the measurement on the CHESS-step student.
Difference-in-means is the optimal linear probe under isotropic noise (Theorem 43).	That residual-stream noise is isotropic. It is not.
Structure tags improve subspace separation (IMAD, ablation).	The size of that effect for CHESS roles specifically.

Remark 46 (a use beyond interpretability)**.** If the six directions are real, steering toward `candidate_generator` with different α is a cheap diversity knob: it recovers some of CHASE-SQL's multi-path candidate generation from a *single* model at one forward pass per candidate. That is a stated motivation in `caa.py` and an untested idea, not a result.

Part VII
The training objective

29 Cross-entropy, from scratch

Training a language model means making the correct next token more probable. The standard way to say that numerically is **cross-entropy loss**.

Definition 47 (Per-token cross-entropy)**.** If the model assigns probability $p_t = p(x_{t+1} = x^\star_{t+1} \mid x_{\le t})$ to the correct next token, the loss at that position is

$$\mathcal{L}_t = -\log p_t.$$

Read it as a surprise meter. If the model was certain and right, $p_t = 1$ and $\mathcal{L}_t = -\log 1 = 0$: no surprise, no loss, nothing to learn. If it was certain and wrong, $p_t \to 0$ and $\mathcal{L}_t \to \infty$. If it had no idea at all among $|V|$ options, $p_t = 1/152064$ and $\mathcal{L}_t = \log 152064 \approx 11.93$.

Example 48 (the numbers to keep in mind)**.**

$\mathcal{L}_t$	p_t	interpretation
11.93	1/152064	uniform guess
1.0	0.368	plausible but unsure
0.1	0.905	confident
0.01	0.990	very confident
0.0008	0.9992	**memorised**

The last row is not hypothetical. It is the measured mean training loss at stage 4, and Part IX is about what it means.

Definition 49 (Masked sequence loss)**.** Given a label vector $z_1, \dots, z_T$ where $z_t = -100$ marks "do not supervise this position",

$$\mathcal{L} = \frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} -\log p(x_{t+1} = z_t \mid \cdot), \qquad \mathcal{S} = \{t : z_t \neq -100\}. \tag{7.1}$$

The average is over supervised positions only.

> $|\mathcal{S}|$ — how many positions are graded — is the quantity to watch. It determines what the loss can and cannot see.

30 The COCONUT sequence loss, exactly as the code computes it

We can now write down the training loss precisely. Fix a training record: a prompt c, a gold SQL $y^\star$ with token ids $(g_1, \dots, g_n)$, and a stage k.

The function `_coconut_sequence_loss` (`latentsql/distill/latent_sft.py`) builds a list of input *embeddings* — not token ids, because latent positions have no token id — together with a parallel label list. In order:

1. **Prompt.** Append $e(c_1), \dots, e(c_{|c|})$, all labelled -100.
2. **For each role r_j, $j = 1, \dots, k$:**

(a) append the embeddings of $\texttt{tag}(r_j)$, labelled -100;

(b) repeat m times: run the model on everything built so far, take the last-layer last-position hidden state h, append $\text{align}(h)$ as the next input embedding, and append the label -100.

3. **Consensus.** Append the embeddings of `<|consensus|>`, labelled -100.

4. **Target.** Append $e(g_1), \ldots, e(g_n)$, labelled $g_1, \ldots, g_n$ — *the only supervised positions.*

One final teacher-forced forward pass over the assembled sequence produces (7.1). Writing Φ_k for the assembled prefix at stage k:

$$\boxed{\mathcal{L}_k^{\text{main}} = -\frac{1}{n}\sum_{i=1}^{n} \log p_\theta\big(g_i \,\big|\, \Phi_k,\ g_{<i}\big)} \tag{7.2}$$

$|\mathcal{S}| = n \approx 62$ for every record, at *every* stage. The prompt is not graded. The tags are not graded. The latent positions are not graded — they cannot be, they have no target. The number of graded positions does not change as the curriculum deepens.

30.1 Why the latent positions get gradients anyway

A natural objection: if latent positions are not supervised, how does anything about them get trained?

Through the chain rule. The gold SQL's probability depends on the state the model is in when it starts decoding, that state is a function of the latent vectors, and each latent vector is a differentiable function of the model's parameters (it is align of a hidden state that the model produced). So

$$\frac{\partial \mathcal{L}_k^{\text{main}}}{\partial \theta} = \underbrace{\frac{\partial \mathcal{L}}{\partial \Phi_k}}_{\text{how the prefix should change}} \cdot \underbrace{\frac{\partial \Phi_k}{\partial \theta}}_{\text{through all } km \text{ latent steps}} + \text{direct paths.}$$

Nothing forces the latent vectors to *mean* anything in particular. They are shaped only by the pressure "make the correct SQL likely at the end". This is Coconut's own framing: the objective "does not encourage the continuous thought to compress the removed language thought, but rather to facilitate the prediction of future reasoning".

Remark 50 (training does $km + 1$ forward passes)**.** The latent steps are sequential — each needs the previous one's output as its input — so a stage-k record costs km forward passes to build the prefix plus one more for the loss. At $k = 6$, $m = 2$ that is 13. This is why training is slow and why the code uses `use_cache=False` in this path: the memory-safer shape at the cost of recomputation.

31 The SQL-readiness probe

Equation (7.2) grades exactly one thing: the SQL, at the very end. The **SQL-readiness probe** is an auxiliary term that grades the same thing at an *earlier* point in the sequence.

Definition 51 (The probe)**.** Fix a role index $j^\star$ (the parameter `SQL_READY_ROLE`) and a token budget κ (`SQL_READY_TOKENS`). During the loop of step 2 above, immediately after finishing role $j^\star$'s latent steps, build a *second*, temporary sequence:

$$\Phi_{j^\star} \,\big\|\, \texttt{<|consensus|>} \,\big\|\, (g_1, \ldots, g_\kappa),$$

supervise only the κ target tokens, and record its loss

$$\mathcal{L}^{\text{probe}} = -\frac{1}{\kappa}\sum_{i=1}^{\kappa} \log p_\theta\big(g_i \mid \Phi_{j^\star}, g_{<i}\big). \tag{7.3}$$

The total objective is

$$\boxed{\mathcal{L}_k = \mathcal{L}_k^{\text{main}} + \lambda\, \mathcal{L}^{\text{probe}}} \tag{7.4}$$

with $\lambda = \texttt{SQL_READY_WEIGHT} = 0.5$.

The intent, in one sentence: *teach the model that* `<|consensus|>` *means "start writing SQL now" regardless of how many latent roles preceded it.*

31.1 A degeneracy: why $j^\star = k$ does nothing

Proposition 52. *If $j^\star = k$ — the probe fires after the* last *latent role of the current stage — then $\mathcal{L}^{\text{probe}}$ is a truncated copy of $\mathcal{L}_k^{\text{main}}$, and (7.4) reduces to a reweighting of the main loss rather than adding any new constraint.*

Proof. The probe's prefix is $\Phi_{j^\star}$ followed by `<|consensus|>`. When $j^\star = k$, $\Phi_{j^\star} = \Phi_k$, which is exactly the main sequence's prefix; the main sequence also continues with `<|consensus|>`. The two sequences are therefore token-for-token and vector-for-vector identical up to that point. The probe then supervises $(g_1, \ldots, g_\kappa)$; the main loss supervises $(g_1, \ldots, g_n)$ with $\kappa \le n$. So $\mathcal{L}^{\text{probe}}$ is the mean of the first κ of the same n per-token terms that $\mathcal{L}_k^{\text{main}}$ averages, computed at the same positions from the same states. It contributes no gradient direction that $\mathcal{L}_k^{\text{main}}$ does not already contain. □

Visually, at stage 4 with $j^\star = 4$:

```
probe :  prompt | r1 oo | r2 oo | r3 oo | r4 oo | <|consensus|> | SQL[:16]
main  :  prompt | r1 oo | r2 oo | r3 oo | r4 oo | <|consensus|> | SQL
         ^--------------- identical ---------------^
```

31.2 Why $j^\star = k - 1$ is not degenerate

Set $j^\star = 3$. At stage 4:

```
probe :  prompt | r1 oo | r2 oo | r3 oo |         <|consensus|> | SQL[:24]
main  :  prompt | r1 oo | r2 oo | r3 oo | r4 oo | <|consensus|> | SQL
                                        ^ probe cuts in one role early
```

Now the probe's prefix is a sequence the main loss never evaluates. In one optimisation step the model is graded on producing the same SQL from *two different prefix lengths*. It cannot satisfy both by memorising one positional pattern; the only representation that satisfies both is the general rule "`<|consensus|>` means SQL, whatever preceded it".

There is a second reason to prefer $j^\star = 3$: it is stage-independent. The probe is active only when $k \ge j^\star$ (the `sql_ready_active` gate in the code), so

$j^\star$	stage 4	stage 5	stage 6
4	degenerate	mid-prefix	mid-prefix
3	mid-prefix	mid-prefix	mid-prefix

With $j^\star = 4$ the probe was a genuine constraint at exactly the two stages that were not broken, and a no-op at the one that was.

Remark 53 (honest note, carried forward)**.** Proposition 52 shows $j^\star = 4$ was the wrong setting. It does *not* show that fixing it is what repaired stage 4. Measured values of $\mathcal{L}^{\text{probe}}$ during the retrain were of order 10^{-4} (Table 1) — as saturated as the main loss — so its gradient contribution was tiny. This is picked up in Section 43.

Part VIII
The optimiser, and one uncomfortable fact about Adam

The loss of Part VII has to be turned into weight updates. How that is done turns out to matter as much as the loss itself, so it gets its own Part.

32 What is actually being trained: LoRA

We do not update all 7×10^9 parameters. We use **LoRA** (Low-Rank Adaptation): for a selected weight matrix $W_0 \in \mathbb{R}^{a\times b}$, freeze it and learn a low-rank correction,

$$W = W_0 + \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{a\times r},\ A \in \mathbb{R}^{r\times b},\ r \ll \min(a,b). \tag{8.1}$$

Only A and B are trainable. Our settings are $r = 32$, $\alpha = 64$ (so the scaling factor is $\alpha/r = 2$), applied to `q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj` in every layer.

Remark 54. LoRA matters here for a reason beyond memory: because W_0 is frozen, whatever the base model already believed is still fully present. It has not been overwritten, only offset. Mechanism 3 of Part IX is exactly a case of the frozen base's prior reasserting itself.

33 Adam, derived

Plain gradient descent takes $\theta \leftarrow \theta - \eta g$ with $g = \nabla_\theta \mathcal{L}$. Its weakness is that a single global η must suit parameters whose gradients differ in scale by orders of magnitude. Adam fixes this by giving each parameter its own effective step size, estimated from its own gradient history.

Definition 55 (Adam)**.** With $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, and moments initialised to zero, at step t:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t && \text{(momentum: a running mean of } g\text{)} \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 && \text{(a running mean of } g^2\text{)} \\ \hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t} && \text{(bias correction)} \\ \theta_t &= \theta_{t-1} - \eta \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}. \end{aligned} \tag{8.2}$$

The bias correction exists because $m_0 = v_0 = 0$ drags the early averages toward zero; dividing by $1 - \beta^t$ removes that bias exactly.

33.1 The normalisation, and what it costs

Look at the update direction in (8.2):

$$\Delta\theta = -\eta \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

$\hat{m}_t$ is an average of g; $\sqrt{\hat{v}_t}$ is (roughly) the root-mean-square of g. Their ratio is *scale-free*: multiply every gradient by 1000 and both numerator and denominator scale by 1000, leaving $\Delta\theta$ unchanged.

Adam does not take smaller steps when the gradient is small. It takes steps of size set by η, in the direction the gradient points, essentially regardless of the gradient's magnitude. A gradient of 10^{-4} moves the weights about as far as a gradient of 10^{-1}.

This is a feature when the small gradient is a genuine, consistent signal. It is a hazard when the small gradient is *noise* — which is exactly the situation when the loss has saturated.

Proposition 56 (the first step after a moment reset is a pure sign step)**.** *Immediately after the moments are reset to zero, the first Adam update is*

$$\Delta\theta = -\eta \, \frac{g}{|g| + \varepsilon} \;\approx\; -\eta \, \mathrm{sign}(g),$$

independently of $\|g\|$.

Proof. Set $t = 1$ with $m_0 = v_0 = 0$. From (8.2),

$$m_1 = (1 - \beta_1) g, \qquad v_1 = (1 - \beta_2) g^2.$$

Bias correction divides by $1 - \beta_1^1 = 1 - \beta_1$ and $1 - \beta_2^1 = 1 - \beta_2$ respectively, which cancels those factors exactly:

$$\hat{m}_1 = \frac{(1 - \beta_1) g}{1 - \beta_1} = g, \qquad \hat{v}_1 = \frac{(1 - \beta_2) g^2}{1 - \beta_2} = g^2.$$

Hence $\sqrt{\hat{v}_1} = |g|$ and

$$\Delta\theta = -\eta \, \frac{g}{|g| + \varepsilon},$$

which for $|g| \gg \varepsilon$ is $-\eta \, \mathrm{sign}(g)$: a step of length exactly η per coordinate, whatever the gradient's size. □

Combine Proposition 56 with the curriculum's design choice of resetting the optimiser at every stage boundary (Section 23) and the consequence is immediate: **the first update of every stage is a full-size step in whatever direction the residual gradient happens to point**, even if that gradient is pure noise from an already-memorised objective.

34 Gradient accumulation, and how few steps there really are

Gradients are accumulated over several records before an update:

```
(loss / cfg.grad_accum).backward()
...
if n_acc % cfg.grad_accum == 0:
    torch.nn.utils.clip_grad_norm_(params, 1.0)
    optim.step()
    optim.zero_grad(set_to_none=True)
```

This is mathematically the gradient of the mean loss over `grad_accum` records — a larger effective batch at constant memory.

The runner sets `--grad-accum "$limit"`, i.e. equal to the batch size. So one process = one batch = *exactly one* optimiser step.

Example 57 (the real step budget)**.** With $N = 94$ usable records, `BATCH_SIZE` $= 16$ and `EPOCHS_PER_STAGE` $= 3$:

$$\left\lceil \frac{94}{16} \right\rceil = 6 \text{ batches per epoch}, \qquad 6 \times 3 = \boxed{18 \text{ optimiser steps per stage}}.$$

Meanwhile $94 \times 3 = 282$ forward/backward passes are logged per stage, which matches the step-record counts exactly.

18 steps is very few. Under Proposition 56 the first of them is a full-η sign step, and with $\beta_1 = 0.9$ the momentum is still far from equilibrium for several more. At $\eta = 5 \times 10^{-5}$ and LoRA's $\alpha/r = 2$ amplification, this is real movement in the adapter, in a direction chosen by whichever one or two of 16 records still had non-trivial loss.

Gradient clipping to norm 1.0 is applied before every step. Note what it does and does not do: it bounds the update when gradients are *large*. It has no effect at all when they are small, which is the regime that causes trouble here.

Part IX
A worked failure: the stage-4 collapse

Everything in Parts VII and VIII was set up so that this Part can be read as a derivation rather than a story. A real curriculum run broke in a specific, reproducible way; the cause turned out to be a consequence of the objective's shape and the optimiser's normalisation, both of which have now been stated precisely.

35 The symptom

The CHESS-step curriculum was run for six stages. After each stage the checkpoint was probed on a fixed 30-question held-out sample.

stage k	train loss (mean)	probe EX (%)	avg output tokens
1	0.1332	66.67	62.7
2	0.0186	66.67	67.8
3	0.0019	63.33	61.0
4	**0.0008**	**26.67**	**125.7**
5	0.0015	56.67	72.4
6	0.0014	63.33	67.4

Stage 4 loses 36.66 points of execution accuracy and doubles its output length.

The project's own earlier documentation described this as "an optimization/representation instability". That was wrong, and the table above is the refutation: **stage 4 has the lowest mean loss and the lowest maximum loss of any stage** (max = 0.1265, versus 0.9836 at stage 1). There is no instability. Training went better at stage 4 than anywhere else.

36 What the failures actually looked like

14 of the 30 stage-4 predictions were unparseable, and all 14 failed the same way — schema-linking prose after the consensus marker, repeated until the 200-token generation cap:

```
<|consensus|>Tables:
Tables: visitor, visit, museum
Tables:
Tables: visitor, visit, museum
... (to the cap)
```

The doubled output length is the same fact seen from the other side: those are not longer queries, they are cap-length loops.

The model has not forgotten SQL. It has forgotten that `<|consensus|>` means "emit SQL". It has reverted to a different, older belief about what follows a role tag.

37 Mechanism 1: the objective is saturated by stage 3

Recall (7.2): $|\mathcal{S}| = n \approx 62$ supervised tokens per record, and $N = 94$ records, run for 3 epochs per stage.

By stage 3 the mean per-token cross-entropy is 0.0019; by stage 4 it is 0.0008, i.e. $p_t \approx 0.9992$ on the correct next token.

> 94 SQL strings, seen 3 times each per stage across six stages, at a per-token loss of 8×10^{-4}. They are **memorised**. There is essentially no learning signal left in $\mathcal{L}^{\text{main}}$.

The reason this happens so completely is the SQL-direct format (Definition 38). Look again at the code:

```
add_text(chat, supervise=False)
for idx, role in enumerate(cfg.roles):
    if role == "consensus":
        break
    if idx >= n_latent_roles:
        break            # <-- latent_sft.py:113
    ...
add_text(T_CONSENSUS_OPEN, supervise=False)
add_token_ids(target_ids, supervise=True)
```

Line 113 `break`s out of the loop for every role past the current stage. It does not fall through to a branch that appends those roles as *text*. There is no such branch on this path. Confirmation from the logs: `text_loss` is `null` in all 1908 step records of the run.

> **This is where LatentSQL departs from Coconut.** Coconut replaces the first k reasoning steps with latents *and keeps the remaining steps as supervised text*, so the supervision set shrinks gradually as k grows. Here the remaining steps are deleted, so the supervision set is a constant ≈ 62 SQL tokens at every stage. The curriculum's difficulty ramps; the objective's difficulty does not.

38 Mechanism 2: Adam steps into the noise at full size

Now apply Part VIII. Per Example 57 there are 18 optimiser steps per stage, and the runner deletes `optim_state.pt` at each stage boundary, so every stage begins from zero momentum.

By Proposition 56, the first step of stage 4 is

$$\Delta\theta = -\eta \operatorname{sign}(g), \qquad \eta = 5 \times 10^{-5},$$

a full-magnitude step regardless of the fact that $\|g\|$ is now tiny. And by the scale-freeness of (8.2), the following steps are not much smaller.

> **The two mechanisms compose badly.** Mechanism 1 says the gradient carries almost no information. Mechanism 2 says Adam will nonetheless move the weights the full distance η in whatever direction that near-zero gradient points. The adapter drifts, driven by whichever one or two of 16 records in a batch still have non-trivial loss.

39 Mechanism 3: where the drift lands

Drift alone would be harmless if the weights had nowhere bad to go. They do.

The curriculum starts from `sft_7b_bf16`, an adapter trained by ordinary SFT on *full written debate transcripts* — schema-linking blocks that literally begin `Tables: ...`, candidate discussions, unit tests. In those transcripts, the thing that follows a role tag is prose.

By (8.1) that belief was never deleted: W_0 is frozen and the LoRA correction merely offsets it. The latent curriculum trains a competing behaviour — "after the tags, emit SQL" — but only on the 94 training records, where the model already gets it right and therefore is never penalised for the old prior. On any held-out question the two behaviours compete, and a few full-size Adam steps in a noise direction are enough to tip the balance.

> This is a **generalisation** failure, not an optimisation one, and it is invisible to the training objective by construction. The objective is evaluated on exactly the distribution where the failure does not occur.

Proposition 58 (the failure is unobservable to the objective)**.** *Let $\mathcal{D}_{\text{tr}}$ be the 94 training records and $\mathcal{D}_{\text{ho}}$ the held-out set. Suppose parameters θ satisfy $\mathcal{L}(\theta;\, x) < \epsilon$ for all $x \in \mathcal{D}_{\text{tr}}$ while the model's greedy continuation after `</consensus/>` on some $x' \in \mathcal{D}_{\text{ho}}$ is not SQL. Then no quantity computed from $\{\mathcal{L}(\theta; x)\}_{x \in \mathcal{D}_{\text{tr}}}$ — its mean, maximum, variance, or trajectory — can distinguish θ from a θ' that generalises correctly.*

Proof. Immediate: the hypothesis fixes the values of every such quantity on $\mathcal{D}_{\text{tr}}$ to be within ϵ of the same numbers for both θ and θ', while the behaviour that differs is evaluated only on $\mathcal{D}_{\text{ho}}$, which those quantities do not depend on. □

This is why the per-stage held-out probe exists in the runner at all, and why it caught something a loss curve never would.

40 The bonus finding: the fix that was already configured did nothing

The run had `SQL_READY_ROLE=4` set, precisely to anchor the "consensus means SQL" rule. By Proposition 52, at stage 4 with $j^\star = 4$ the probe is a truncated duplicate of the main loss.

> The auxiliary constraint was a genuine mid-prefix anchor at stages 5 and 6 — which were never the problem — and a no-op at stage 4, which was.

41 The repair

Two changes, applied from stage 4 onward, resuming from the stage-3 checkpoint into a fresh run directory (`chess_coconut_sql_direct_s4fix`) so the original run and the downstream GRPO adapter were left untouched:

knob	before	after	targets
learning rate, stage ≥ 4	5×10^{-5}	1×10^{-5}	mechanism 2
`SQL_READY_ROLE`	4	3	mechanism 3
`SQL_READY_WEIGHT`	0.5	0.5	—
`SQL_READY_TOKENS`	16	24	—

The learning-rate change reduces the size of every drift step by 5×. The probe change makes (7.4) a real constraint at stages 4, 5 and 6 (Section 31.1).

Remark 59 (what was deliberately not changed)**.** The obvious structural fix — restore Coconut's remaining-steps-as-text supervision, so that $|\mathcal{S}|$ grows with the curriculum and the objective never saturates — was *not* attempted. It changes the format the downstream GRPO stage and the evaluation harness both assume. It is recorded as the next lever if a hyperparameter fix had failed.

42 Result of the retrain

stage	EX old	EX new	unparseable	hit 200-cap	out tokens
4	26.67	**66.67**	14 → **0**	14 → **0**	125.7 → **61.6**
5	56.67	**66.67**	3 → **0**	2 → **0**	72.4 → **61.2**
6	63.33	63.33	0 → 0	1 → **0**	67.4 → **62.2**

The collapse is gone (+40.00 EX at stage 4) and the `Tables:` loop is eliminated outright. Note also what the table reveals about the *original* run: stage 5's apparent "recovery" was never complete — it still carried 3 unparseable predictions and a 72.4-token average. Stages 5 and 6 had been partly spending their capacity undoing stage 4's damage.

On the full held-out set:

run	n	EX	bird	spider	out tok	unparseable	latency
stage 6, original	90	52.22	33.33	71.11	67.5	4	2.212 s
stage 6, **retrained**	90	**57.78**	**37.78**	**77.78**	**61.2**	**0**	**2.073** s

+5.56 EX, improving on *both* benchmarks, output length down 9.3%, latency down 6.3%, every unparseable prediction eliminated.

Remark 60 (probe optimism)**.** The $n = 30$ stage-6 probe is flat between the two runs (63.33 both times); the advantage appears only on the full $n = 90$ set. A 30-question probe has a standard error of roughly 9 points, so it can rank two nearby checkpoints incorrectly. It is a smoke detector, not a scoreboard.

43 Training loss during the retrain, and the attribution caveat

stage	steps	$\mathcal{L}^{\text{main}}$ mean	$\mathcal{L}^{\text{main}}$ max	$\mathcal{L}^{\text{probe}}$ mean
4	311	0.00030	0.0252	2.39×10^{-4}
5	282	0.00118	0.3388	1.44×10^{-3}
6	282	0.00003	0.0001	8.15×10^{-6}

Table 1: Retrained stages. Both loss terms are saturated throughout.

Two knobs moved at once. This is not a clean ablation.

The evidence favours the learning rate doing nearly all the work: $\mathcal{L}^{\text{probe}}$ logged $\sim 10^{-4}$ throughout, i.e. the auxiliary term was as saturated as the main one and contributed almost no gradient.

Supported claim: lowering the late-stage learning rate fixes the stage-4 collapse.
Not supported: the SQL-readiness probe fixes it.

A clean separation requires a third run at $\eta = 10^{-5}$ with `SQL_READY_WEIGHT`$= 0$. It has not been run.

Note also that Table 1 independently confirms Mechanism 1: at the *lower* learning rate the losses are, if anything, even smaller. A saturated objective was never the thing the learning rate fixed — what it fixed was how far the weights travelled while the objective was blind.

44 Two infrastructure failures worth recording

Both occurred at a batch boundary, and both trace to one fact: *a process exiting does not mean its CUDA context has been released.*

1. **Out of memory.** Two per-batch processes overlapped (17.47 + 5.91 GB against a 22.57 GB budget) and the run aborted — even though the batch's own peak was only 18.5 GB. Fix: a `gpu_settle` helper that polls `nvidia-smi` until the card has drained below 1500 MiB before each batch and each evaluation.

2. **A silent CPU downgrade, which was worse.** The next attempt logged *zero* training steps in 48 minutes while burning 1h53m of CPU time at 18.3 GB resident, with the GPU idle at 40 MiB. `torch.cuda.is_available()` had returned `False` transiently, `resolve_device("auto")` therefore returned `"cpu"`, and a 7B model in `bfloat16` trained happily in host RAM roughly 500× too slowly *without raising anything.*

An out-of-memory error at least fails loudly. A silent device downgrade produces a "successful" run, exit code 0, days later, with a checkpoint trained on nothing. The fix is a guard that converts the silent fallback into an immediate error, shown below; the runner sets `LATENTSQL_REQUIRE_CUDA=1` on every batch and every evaluation, and retries each batch once after 20 s, since both failure modes clear within seconds.

```
if torch.cuda.is_available():
    return "cuda"
if os.environ.get("LATENTSQL_REQUIRE_CUDA", "").strip() not in ("", "0", "false", "False"):
    raise RuntimeError(
        "LATENTSQL_REQUIRE_CUDA is set but torch.cuda.is_available() is False - "
        "refusing to fall back to CPU.")
```

Part X
Reinforcement learning: internalisation with GRPO

Curriculum SFT (Parts VII–IX) teaches the model to *produce* a latent-then-SQL sequence that imitates the teacher's structure. It never once optimises for *correctness* directly, and it never rewards brevity. That is IMAD's stage 2.

45 Group Relative Policy Optimisation, from the policy-gradient theorem

Definition 61 (The objective)**.** Let $\pi_\theta(y \mid x)$ be the probability the current model assigns to generating completion y given input x. Training wants to maximise expected reward,

$$J(\theta) = \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\big[R(x,y)\big] = \sum_y \pi_\theta(y \mid x)\, R(x,y).$$

Theorem 62 (Policy gradient)**.**

$$\nabla_\theta J(\theta) = \mathbb{E}_{y\sim\pi_\theta}\big[\nabla_\theta \log \pi_\theta(y \mid x)\cdot R(x,y)\big].$$

Proof. Since R does not depend on θ, $\nabla_\theta J = \sum_y \nabla_\theta \pi_\theta(y \mid x)\, R(x,y)$. Using the log-derivative identity $\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$ (chain rule on log), this becomes $\sum_y \pi_\theta(y \mid x)\, \nabla_\theta \log \pi_\theta(y \mid x)\, R(x,y)$, which is by definition an expectation under π_θ. □

This is what makes RL on a language model tractable at all: the intractable sum over every possible SQL query is replaced by sampling a handful of actual completions and averaging.

Proposition 63 (a baseline is free)**.** *For any b not depending on y, $\mathbb{E}_{y\sim\pi_\theta}[\nabla_\theta \log \pi_\theta(y \mid x)\cdot b] = 0$.*

Proof. The left side equals $b \cdot \nabla_\theta \sum_y \pi_\theta(y \mid x)$. But $\sum_y \pi_\theta(y \mid x) = 1$ identically (it is a probability distribution), and the derivative of a constant is 0. □

So subtracting any y-independent quantity leaves the expected gradient unchanged while reducing its variance — which licenses GRPO's construction.

Definition 64 (GRPO advantage)**.** For a group of G rollouts $y_1, \ldots, y_G$ sampled at the same x, with $r_i = R(x, y_i)$,

$$A_i = \frac{r_i - \bar{r}}{s}, \qquad \bar{r} = \text{mean}(r_1, \ldots, r_G), \quad s = \text{std}(r_1, \ldots, r_G). \tag{9.1}$$

Corollary 65 (a saturated group has zero gradient)**.** *If $r_1 = \cdots = r_G$, then $A_i = 0$ for every i, and the resulting gradient estimate $\frac{1}{G}\sum_i A_i \nabla_\theta \log \pi_\theta(y_i \mid x)$ is exactly the zero vector.*

Proof. $\bar{r} = r_1$ makes every numerator $r_i - \bar{r} = 0$, so every $A_i = 0$ regardless of how $s = 0$ is handled. □

This is not a hand-wave — it is exactly what `grpo.py` logs as `adv_nonzero_frac` $= 0$: a group where every sampled rollout agrees on reward (all correct, or all wrong) contributes *nothing* to that step. It is the single most common failure mode this project's own diagnostics recorded for GRPO.

46 The clipped surrogate and the KL penalty

The code (`grpo.py`) does not apply (9.1) directly to $\log \pi_\theta$; it uses PPO's clipped surrogate, with importance ratio $\rho = \exp(\log \pi_\theta(y \mid x) - \log \pi_{\theta_{\text{old}}}(y \mid x))$ (here $= 1$ on the very step it is sampled from, since $\theta = \theta_{\text{old}}$ at that instant):

$$\mathcal{L}_i^{\text{PG}} = -\min\big(\rho_i A_i,\ \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon)\, A_i\big), \qquad \text{KL} \approx e^{\delta} - \delta - 1, \quad \delta = \log \pi_\theta - \log \pi_{\text{ref}},$$

$$\mathcal{L} = \mathcal{L}^{\text{PG}} + \beta_{\text{KL}} \cdot \text{KL}, \qquad \epsilon = 0.2, \quad \beta_{\text{KL}} = 0.02.$$

The clip bounds how far one step can trust a single sample; the KL term (an always-non-negative estimator of the divergence from a frozen reference policy) discourages the policy from drifting arbitrarily far in pursuit of reward alone — exactly the kind of unconstrained drift Section 56 showed Adam is willing to take.

47 IMAD's reward, specialised to SQL execution

IMAD's reward (its eq. 1) is

$$r(x, y) = w_{\text{fmt}}\, R_{\text{fmt}} + w_{\text{clip}}\, R(y; l), \qquad R(y; l) = \begin{cases} 1 & y^\star \text{ appears in the first } l \text{ tokens of } y \\ 0 & \text{else.} \end{cases} \tag{9.2}$$

Two schedules anneal over training:

$$w_{\text{fmt}} : 1.0 \to 0.05, \qquad l : l_0 \to l^\star.$$

The mechanism, in one sentence: the decaying format reward removes the incentive to *verbalise* the debate; the shrinking length limit makes verbalising it *impossible* while still earning the correctness reward. The only strategy left is to do the multi-perspective analysis internally and answer directly.

For Text-to-SQL, "correct" is redefined as execution match (Definition 1) rather than string containment, so $R(y; l)$ actually runs the clipped SQL against the database. Two shaping terms are added, small so IMAD's two terms still dominate: $w_{\text{exec_ok}}$ (the query parses and runs) and $w_{\text{tag_order}}$ (structure tags in the canonical order).

47.1 Dense partial credit, for when GRPO would otherwise learn nothing

Corollary 65 is not hypothetical here: a rollout group that is *entirely wrong* has zero variance under (9.2), and by Proposition 63/Corollary 65 contributes no gradient. Two extra terms, awarded *only to incorrect rollouts*, break the tie between a near-miss and a badly-wrong one:

$$w_{\text{atomic}} \cdot R_{\text{atomic}}(y, y^\star) \ + \ w_{\text{memory}} \cdot R_{\text{memory}}(y)$$

where R_{atomic} scores atomic-SQL-operator overlap with the gold query and R_{memory} scores similarity to previously-verified reasoning traces (annealed on w_{fmt}'s schedule, so it fades exactly as the incentive to reason in prose fades).

47.2 Brevity, for when a group is entirely correct

The mirror problem: a group where every rollout is already correct *also* has zero variance, and — since the entire purpose of internalisation is to stop verbalising — this is precisely the case where GRPO most needs to say something. Among correct rollouts only,

$$R_{\text{brevity}}(y) = \max\Big(0,\ 1 - \frac{|y|}{\text{ref}}\Big),$$

so a terse correct answer outscores a verbose correct answer, giving the group non-zero variance again and restoring a gradient.

47.3 Spurious correctness: when a group is entirely correct, but shouldn't be

Brevity fixes one way an all-correct group can be a trap: every rollout is right, so there is nothing to teach from. There is a second, nastier way the *same* group can look all-correct without actually being so — and this one is not about length at all.

Step 1: the problem, with the smallest possible example. Definition 1 checks one thing: does the predicted query return the same rows as gold, *on the one database the benchmark ships*. That database is a single, fixed snapshot of the data. Two queries that mean completely different things can still agree on it, purely by accident of what values happen to be sitting in the table that day.

Example 66 (two different queries, one lucky database)**.**

```
Question:   Which employees work in the Research department?
Gold:       SELECT name FROM employee WHERE dept = 'Research';
Candidate:  SELECT name FROM employee WHERE salary BETWEEN 50000 AND 60000;
```

Suppose, on today's data, every Research employee happens to earn between $50,000 and $60,000, and nobody else does. Both queries return the same two names. Definition 1 scores the candidate 1 — correct. It is not correct: it says nothing about departments at all, and would return the wrong names the moment a new employee is added with a salary in that range but a different job.

Why this is an RL problem, not just an eval problem. In GRPO, the reward *is* R_{clip} (eq. 9.2). If a whole group of G rollouts each independently stumbles onto a different accidentally-matching query, the group looks exactly like the saturated, all-correct case Brevity was built for (Corollary 65: zero variance, nothing to learn) — except here the group isn't actually all correct. Left alone, GRPO has no way to tell a robustly-correct rollout from a lucky one, and nothing stops it from reinforcing the lucky kind just as happily as the real kind.

Step 2: the published fix, and the one piece of it that does not transfer. A project document kept locally (`pool_text2sql_crowd_source_mixture/main_pool_eval.pdf`, "PoolEvaluator", Appendix B, "EX and EX-Ext", anonymous submission under review at ICLR 2027) names exactly Example 66 and fixes it for *offline evaluation*: build several modified copies of the database, require gold and the prediction to agree on *every* copy, not just the original. Their Table 2 does precisely this to Example 66's pair — one modified copy adds a Research employee earning outside $50,000–$60,000, another adds a non-Research employee earning inside that range, and the candidate fails both, correctly exposing it as wrong.

Their method picks *which* row to add using an LLM (their GPT-5.4), *per query pair* — it already knows both the gold and the candidate query when it designs the row. That does not transfer to a reward function: GRPO calls the reward on a brand-new, unforeseen rollout at every single training step, so there is no query pair to hand-design a row for in advance. Only the *gold* query is known ahead of time. The principle (check agreement across several databases, not one) transfers; the mechanism (an LLM designs each database) has to be replaced by something automatic that only ever looks at gold.

Step 3: three automatic ways to build those extra databases. All three take only the gold query as input, so they can run once, offline, before training ever starts — the same stage as building the value index (Part V's `information_retriever` role) — rather than once per rollout.

Method A — plant the missing counterexample. Parse gold's own `WHERE`/`HAVING` conditions and read off every *(column, comparison, literal)* triple: in Example 66, `(dept, =, 'Research')`. For each one, insert two new rows — one that satisfies it, one that just misses it — filling every other column with a real sampled value so the row looks ordinary. This is exactly what manufactures Table 2's two counterexample rows, with no human or LLM choosing where to put them: the boundary comes straight out of gold's own literal.

Method B — keep only the instances that actually test something. Not every inserted row is useful; filter by trying a handful of tiny one-edit changes to *gold itself* (flip `=` to `!=`, nudge a number by 1, swap `AND`/`OR`) and keeping an instance from Method A only if at least one of those edited versions of gold disagrees with gold on it. This is the automatic stand-in for the paper's LLM judgement call — the same question ("does this database separate gold from something plausible?"), answered by trying plausible edits instead of asking a model.

Method C — shake what already exists. No parsing, no reading of gold's literals at all: copy the database, then delete or duplicate a handful of its *existing* rows at random.

Example 67 (what Method C can and cannot catch)**.** Suppose gold correctly counts distinct singers at one stadium, `COUNT(DISTINCT singer_id)`, and the candidate forgot `DISTINCT`, `COUNT(singer_id)`. If no singer in today's data happens to perform there twice, the two counts agree — another spurious match. **Duplicating one existing link row** breaks this: gold's `DISTINCT` still counts that singer once; the candidate now counts them twice. Caught, with no idea in advance that "missing `DISTINCT`" was the risk.

Now take Example 66 instead. Every row that exists today already satisfies *both* filters or *neither* — there is no existing Research-but-low-salary row to duplicate or delete, because it never existed in the data to begin with. Reshuffling existing rows cannot reveal the two filters are different; only *inserting* the missing row (Method A) can. **Method C is good at multiplicity bugs — duplicate rows, a missing `DISTINCT`, a join that fans out — and blind to filter/threshold bugs**, which is exactly why it is the fallback, not the main method: used only when `sqlglot` is unavailable, or gold has no `WHERE`/`HAVING` literal for Method A to anchor on at all (a bare `SELECT COUNT(*) FROM t`).

Step 4: turning agreement-across-instances into a reward term. Let $\mathcal{I}(y^\star)$ be the set of stress instances built from gold $y^\star$ by Methods A–C (up to some fixed count, e.g. 5, matching the paper's own choice). Define

$$R_{\text{spurious}}(y) \;=\; \frac{1}{|\mathcal{I}(y^\star)|} \sum_{D \in \mathcal{I}(y^\star)} \mathbb{1}\big[\text{exec}(y, D) = \text{exec}(y^\star, D)\big], \tag{9.3}$$

the fraction of stress instances on which the prediction still agrees with gold — a dense score in $[0, 1]$, not a binary pass/fail, exactly like the other two shaping terms in this Part. Awarded, like brevity, *only* to rollouts that already passed the original check ($R_{\text{clip}} = 1$): a rollout that is wrong on the shipped database needs no further test, it is already being handled by §47.1's dense partial credit. The total reward (extending eq. 9.2) gains one more term,

$$r(x, y) \mathrel{+}= -w_{\text{spurious}} \cdot \big(1 - R_{\text{spurious}}(y)\big), \qquad \text{computed only when } R_{\text{clip}}(y) = 1, \tag{9.4}$$

a penalty that is exactly 0 when the prediction survives every stress instance (fully robust — no change in behaviour from before this section) and grows toward $-w_{\text{spurious}}$ the more instances expose it as lucky rather than correct.

What this buys, stated precisely. It does not change what counts as correct on the original database — R_{clip} is untouched. It changes what a GRPO group looks like *after* all of today's queries are already right under R_{clip}: a group that used to be uniformly 1.0 (dead, by Corollary 65) can now split into genuinely robust rollouts and spuriously-lucky ones, restoring exactly the kind of within-group disagreement GRPO needs to have anything to learn from — the same disagreement brevity restores for length, now restored for semantic robustness instead.

Status: implemented, not yet measured. Methods A–C and the reward term above are built (`latentsql/db/spurious.py`, `rewards.py`'s `spurious_agreement`, `w_spurious` in `RewardConfig`), gated behind `w_spurious=0` by default so existing behaviour is unchanged unless explicitly turned on. No GRPO run with it enabled has been made yet. Everything in this subsection is therefore a *mechanism*, reported the same way Part XI reported Dropout-GRPO's and Latent-GRPO's fixes — real, cited, code-backed, but not yet a measured result for this project.

48 Latent rollouts

With `latent_rollouts: true`, GRPO samples SQL not from a plain-text prompt but from the latent prefix of Part V — the KV cache already primed by the role tags and their latent steps — and backpropagates *through* those latent steps, exactly as in curriculum SFT (Section 17's checkpointing discussion applies identically here). When this is active, there is no debate text to anneal away, so $w_{\text{fmt,start}} = w_{\text{fmt,end}} = 0$ and correctness alone drives the reward.

49 What GRPO measurably did

Fifty GRPO steps, fifty hard prompts, brevity enabled, starting from the *original* (pre-repair) stage-6 checkpoint, evaluated on the same held-out $n = 90$ mix:

	SFT (pre-GRPO)	after GRPO	Δ
EX ($n = 90$)	52.22%	**58.89%**	+6.67
bird EX	33.33%	**37.78%**	+4.45
spider EX	71.11%	**80.00%**	+8.89
avg output tokens	67.5	**61.9**	−5.6
unparseable	4	**0**	−4

Accuracy rose while output length fell — the brevity reward doing exactly what it was shaped to do — and every one of the 90 generations stayed parseable, so the latent → `<|consensus|>` → SQL format survived GRPO intact. The gain concentrates on Spider; GRPO's training prompts were drawn from the same length-filtered pool as the eval split, so the Spider number is the more independently-earned half of this result.

These two levers must not be added. GRPO's +6.67 was measured starting from a stage-6 checkpoint whose curriculum *had* the stage-4 collapse in its history (Part IX). The repaired curriculum alone reaches 57.78% EX with *no* GRPO at all (Table in §IX's result section) — nearly the same place, from a different lever. The correct next experiment, not yet run, is GRPO starting from the repaired stage-6 adapter.

Part XI
Does GRPO sharpen the agent subspaces, or not?

Part VI showed SFT builds separable per-agent directions. Part X added a second stage on top: GRPO. This Part asks the question those two leave open, and answers it the same way the rest of this document does: build it from one small fact, then check that fact against this project's own code and four papers read for this section (`pdfs/rl/`).

50 The one fact everything else follows from

Recall two things already proved earlier in this document.

Fact 1 (Definition 64). GRPO grades a *group* of G attempts at the same question and compares them to each other: $A_i = (r_i - \bar{r})/s$.

Fact 2 (Corollary 65). If all G attempts score the same, every $A_i = 0$, and the gradient is exactly zero. **No disagreement inside the group $\Rightarrow$ nothing is learned.**

So the whole question "does GRPO reshape the latent thoughts?" reduces to one much smaller question:

When GRPO samples G attempts at one question, do the G latent thoughts actually differ from each other?

If yes, GRPO has something to compare and can push the latent thoughts apart or together. If no, it cannot — by Fact 2, with nothing to be zero exactly.

51 A die you roll versus a die you average

Here is the entire idea, before any formula. Take a fair six-sided die.

- **Roll it twice.** You might get 3, then 5. Two different numbers — *disagreement*, something to compare.
- **Compute its average twice.** $(1+2+3+4+5+6)/6 = 3.5$. Then do the same arithmetic again: 3.5. **Every single time, no exceptions.** An average has no randomness in it at all — it is one fixed number, not a draw from six possible ones.

Rolling produces disagreement. Averaging never does, no matter how many times you repeat it. This is not a subtle statistical point — it is just what the word "average" means.

Now recall equation (4.3), this project's own recipe for a continuous thought:

$$\text{align}_{\text{soft}}(h) \;=\; \sum_{v \in \mathcal{K}} \pi_v \, W_{\text{in}}[v, :].$$

Read the right-hand side out loud: it is a weighted *average* of 32 embedding rows. Not a roll. An average. Two independent facts confirm this is not a coincidence of wording:

1. **The identical formula appears, independently, in a 2026 GRPO paper.** Deng et al., *Latent-GRPO* (arXiv:2604.27998, their eq. 1), define a latent token exactly the same way:

$$s_t \;=\; \sum_{i=1}^{K} p_{t,i}\, e_{t,i},$$

a weighted average of the top-K embeddings by their softmax probability — the same construction as our (4.3), reached independently for the same reason (efficiency, no extra teacher forward pass). Two unrelated projects landing on the same formula is good evidence it is the natural default, not evidence that it is stochastic — it still is not.

2. **Nothing in `_latent_steps` or `align_soft` draws a random number.** Read the code (Part IV): h goes in, one fixed vector comes out. Run it on the same h a thousand times, get the same vector a thousand times.

This is the whole answer, stated plainly. Our continuous thought is the "average die" from the analogy above, not the "rolled die". Sample a GRPO group of G attempts at the same question with the same weights, and the G latent thoughts are the same computation on the same input, G times — so they come out identical, every time, for the same reason $3.5 = 3.5$.

51.1 Turning that into a one-line proof

Now state it as a proof, so "the same reason $3.5 = 3.5$" is not left as a slogan. Write $\Phi_k(\theta, x)$ for the latent prefix — the state after k role tags and their latent steps (Part V) — and write $\Phi_k(\theta, x)_i$ for the copy of it that rollout i of a GRPO group actually used.

Proposition 68 (the GRPO group shares one latent trajectory)**.** *For every rollout $i = 1, \ldots, G$ in one GRPO step, at one prompt x and one θ: $\Phi_k(\theta, x)_i = \Phi_k(\theta, x)$, identically. The only random variable that differs across i is the decoded completion y_i.*

Proof. Φ_k is a pure function of (θ, x) — no random draw anywhere inside it (§51, point 2). `grpo.py::_sample_group_latent` (l.236–299) computes it *once* and `_clone_caches` the result into all G rollouts (l.269–271); a clone of a fixed value is that same value, G times. `grpo.py::_latent_loss_inner` (l.331–369) instead *recomputes* it once per rollout via `build_prefix(...,grad=True)` (l.338–341) — but every rollout in one step shares the same θ (no `optim.step()` has run yet) and the same x, so a pure function fed the same input twice returns the same output twice: $3.5 = 3.5$, again. Either code path, Φ_k is constant in i; only y_i, built from each rollout's own `completion_ids`, is not. □

Corollary 69 (the latent-path gradient is blind to plan quality)**.** *The gradient of the latent-path loss with respect to the parameters that produce Φ_k can only be informed by comparing completions $y_1, \ldots, y_G$ conditioned on* one shared Φ_k *— never by comparing Φ_k against a different latent trajectory for the same x.*

Proof. By Proposition 68, Φ_k does not vary with i. The only i-dependent terms in the loss are the advantage A_i and $\log \pi_\theta(y_i \mid \cdot)$, both functions of y_i conditioned on the shared Φ_k; there is no term built from two different values of Φ_k for the gradient to use. □

52 Two published fixes, and their actual formulas

If a rolled die gives disagreement and an averaged die does not, the obvious fix is: **stop averaging, start rolling.** Two 2026 papers do exactly this, in two different places, and both prove their fix is mathematically sound rather than just plausible.

52.1 Fix 1 — roll inside the formula (Soft-GRPO / Latent-GRPO)

Deng et al. (arXiv:2604.27998, building on Zheng and Lee's Soft-GRPO) add **Gumbel noise** ξ_i to the log-probabilities *before* the softmax that builds the mixture weights:

$$g'_{t,i} = \log p_{t,i} + \xi_i, \qquad \alpha_{t,i} = \frac{\exp(g'_{t,i}/\tau_g)}{\sum_{u=1}^{K} \exp(g'_{t,u}/\tau_g)}, \qquad s_t = \sum_{i=1}^{K} \alpha_{t,i}\, e_{t,i}. \tag{10.2}$$

Because ξ_i is freshly random for every rollout, $\alpha_{t,i}$ — and therefore s_t — now differs from rollout to rollout, even for the same h: a rolled die, not an averaged one. Crucially, this is not just noise bolted on for its own sake: the Gumbel perturbation has a known, exact log-density (their eq. 3),

$$\log \pi_\theta(s_t) = \sum_{i=1}^{K} \Big[-(g'_{t,i} - \log p_{t,i}) - \exp(-(g'_{t,i} - \log p_{t,i})) \Big],$$

which is exactly what a policy-gradient method needs — the same role $\log \pi_\theta(y_{t+1} \mid \cdots)$ plays for an ordinary sampled token (Part X). With this in hand, the latent step can be treated by the *same* clipped-PPO-ratio machinery Part X already derived for text tokens, just piecewise-defined across the latent segment and the decoded segment (their eq. 5). Latent-GRPO then adds three further, narrower fixes on top of this base construction — *Invalid Sample Advantage Masking* (drop truncated/off-manifold rollouts from the group statistics before normalising), *One-sided Noise Sampling* (only allow $\xi_i \geq 0$, so a positive-advantage rollout's probability mass is never accidentally pushed the wrong way), and *Optimal Correct-Path First-Token Selection* (avoid averaging together two different correct latent paths into one invalid blended state) — because unconstrained two-sided noise on its own is unstable in practice; the base idea, however, is just: **put a real random draw inside the formula that builds the latent token.**

52.2 Fix 2 — roll the weights instead of the output (Dropout-GRPO)

Jung (arXiv:2606.10184) takes the opposite route: instead of perturbing the *output* of each latent step, perturb the *weights* that compute it. Draw one Bernoulli dropout mask ξ per rollout and hold it fixed across all T latent recurrence steps of that rollout:

$$h_t = f_{\tilde{\theta}(\xi)}(h_{t-1}), \qquad t = 1, \ldots, T, \tag{10.3}$$

where $\tilde{\theta}(\xi)$ is the dropout-perturbed weights and f_θ is one transformer block. Because ξ is drawn afresh per rollout, h_T (and so the latent prefix) genuinely differs across the G rollouts of a group — again, a roll rather than an average. Jung then proves, rather than just argues, that this is the right thing to do:

Theorem (Jung 2026, Theorem 3, restated). With the mean-only advantage $A^{(k)} = r^{(k)} - \bar{r}$ and the mask replayed at update time,

$$\mathbb{E}\big[\nabla_\theta \mathcal{L}_{\text{surr}}(\theta)\big]_{\theta=\theta_{\text{old}}} = \Big(1 - \tfrac{1}{K}\Big)\, \nabla_\theta J(\theta)\Big|_{\theta_{\text{old}}},$$

where $J(\theta)$ is the expected reward of the *mask-averaged* policy. The surrogate gradient is an unbiased estimate of the true gradient, up to a known, constant scale factor $(1 - 1/K)$ — not an approximation with unknown error.

On GSM8K they report a Coconut baseline improving from 27.29% to 29.01% pass@1 once this fix is applied — direct evidence that giving the latent step somewhere to disagree turns GRPO from stalled into working, on a Coconut-style model closely related in spirit to this project's own latent mechanism.

53 Neither fix is in this project's code

Neither (10.2) nor (10.3) appears anywhere in `latentsql/`. The latent step (Part IV) is exactly $\text{align}_{\text{soft}}(h)$, computed once, with no injected randomness of any kind. So the averaged-die picture of §51 is the literal, exact situation here, not an analogy that only approximately applies — confirming Proposition 68 and Corollary 69 a second, independent way: not just by reading the sampling code, but by checking the mechanism against the two published recipes for the alternative and finding neither one present.

In one sentence: whatever this project's GRPO run changed, it did not change it by comparing one plan against a different plan for the same question — because every rollout in every group was handed the exact same plan.

54 So why did our own run still move the numbers?

Part X measured +6.67 EX from GRPO. If the latent step never disagrees within a group, where did that gradient come from?

From the one place that still rolls: the SQL decode. After the shared latent prefix, this project's own `_sample_group_latent` (l.276–291) samples the actual SQL tokens with `torch.multinomial` at `temperature=1.0` — a real die roll, every rollout, every token. Two rollouts starting from the *identical* latent state can still decode to two *different* queries, one correct and one not, and Fact 2 (§51.1) needs exactly that kind of disagreement to produce a nonzero gradient. So the group's advantage was not forced to zero the way Jung's Coconut baseline was — but the resulting gradient could only ever be shaped like "write better SQL from this plan", never "pick a better plan." This is a plausible mechanism consistent with the measured numbers, not a controlled test of it: no run at `temperature=0` has been made for comparison.

55 A separate question: did SFT even need to *create* the subspaces?

One more complication, about Part VI rather than about GRPO. IMAD's own number: base-model steering AUC = 0.802, not ≈ 0.5 (chance). That is already well above chance *before* any internalisation. Two 2026 papers, read for this section, suggest why that might be:

- **Persona Vectors** (Chen, Arditi et al., arXiv:2507.21509) extract trait directions by the exact same difference-in-means recipe as Definition 40, and find that a model's finetuning-induced *behaviour* shift on a trait correlates strongly with its finetuning-induced *activation* shift along that trait's own vector ($r = 0.76$–0.97 across traits tested) — finetuning genuinely moves activations along directions a linear probe already finds.
- **Tracing Persona Vectors Through Pretraining** (Moskvoretskii et al., arXiv:2605.13329) trace *when* such directions first appear, and find they form *"within* 0.22% *of OLMo-3 pretraining"* — almost the very start — and remain usable through the entire post-trained model.

The honest, revised claim. "Internalisation creates agent subspaces from nothing" overstates it. "Internalisation makes directions — which may already exist from pretraining — more reliably separable and steerable" is what $0.802 \to 0.903$ and the two papers above jointly support. Theorem 43 is unaffected either way: difference-in-means is still the optimal linear probe for whatever separation exists, whenever in training it first appeared.

56 Summary

Proven here	**Cited, correctly scoped**	**Open**
The GRPO group shares one latent trajectory — proved by reading `grpo.py` (Proposition 68) and by checking that neither published randomness-fix is present (§53).	Two published, proved fixes exist: Gumbel noise in the formula (Latent-GRPO, eq. 10.2), or a shared dropout mask in the weights (Dropout-GRPO, eq. 10.3, exact $(1 - 1/K)$ unbiasedness).	Whether `05_steering.py` run before/after `03_grpo.py` would show the per-role subspaces sharpening, unchanged, or degrading.
By Fact 2, this means the latent-path gradient can never compare two plans for the same question — only two completions of the same plan.	IMAD's base-model AUC (0.802) is high enough to be consistent with pretraining-era structure the persona-tracing paper documents.	Whether applying either published fix to this project's own latent path would change training stability or the resulting geometry.

57 A sibling project, and a genuinely different fix to a shared problem: SERL-SQL

One more comparison is worth making precisely, against a 2026 Text-to-SQL GRPO paper read specifically for this section: **SERL-SQL** (Liu, Feng, Li et al., arXiv:2608.00485, "Selective Hindsight Distillation for Text-to-SQL Reinforcement Agentic Learning"). It targets the exact coarseness this document's own GRPO derivation has (Definition 64), but fixes it a completely different way, in a completely different architecture.

57.1 The same coarse-credit problem, independently named

Recall our own advantage (Definition 64): one scalar A_i per rollout, and recall from (9.1)/§X that this single number is *broadcast identically* onto every token of that rollout's loss (`per_token = pg`

`+ kl_beta*kl`, with `adv` the same A_i at every position). SERL-SQL names this exact structure as its starting complaint, in their own notation (A^n their per-rollout advantage, m, n indexing token/trajectory):

> *"Although this objective provides a stable reward-aligned update direction, its credit assignment remains coarse: decisive SQL actions, reasoning tokens, and formatting tokens receive the same trajectory-level advantage."*
> *"In standard trajectory-level GRPO, this reward-derived advantage is assigned to every action step in the trajectory, i.e. $A_t = A^n$."*

That second sentence is, symbol for symbol, exactly what our own `adv` broadcast does. Two independent projects, applying GRPO to the same task (Text-to-SQL), diagnosed the identical structural fact.

57.2 Two different fixes to the same complaint

This document's own fix (Part XI) targets a *different* symptom of trajectory-level advantage: that every rollout in a group also shares one *latent* prefix (Proposition 68), so the gradient cannot compare two plans, only two completions of one plan. SERL-SQL's architecture has no latent prefix at all — everything is decoded text — so it instead targets the *token-level* symptom directly: give different tokens *within one already-decoded trajectory* different credit.

Their mechanism: a second, *teacher* policy π_T re-scores the student's own tokens *after* seeing execution feedback the student did not have at rollout time (the actual query result, or the actual error message) — information available only in hindsight. The gap between teacher and student log-probability at each token,

$$\Delta_{t,i} = \log \pi_T(\cdot) - \log \pi_\theta(\cdot),$$

is converted to a bounded, sign-aware reweighting factor

$$w_{t,i} = \mathrm{clip}\big(\exp(\mathrm{sgn}(A^n)\,\mathrm{sg}[\Delta_{t,i}]),\ w_{\min},\ w_{\max}\big),$$

applied *only* to SQL/tool-action tokens via a mask $m_{t,i}$ (formatting and reasoning tokens keep weight 1):

$$\bar{w}_{t,i} = (1 - m_{t,i}) + m_{t,i}\, w_{t,i}, \qquad \widetilde{A}_{t,i} = A^n\big[(1-\alpha_k) + \alpha_k\, \bar{w}_{t,i}\big],$$

with α_k decaying so the teacher's influence fades over training. The sign-preserving clip keeps the update direction fixed by A^n while the *magnitude and locality* of credit is reshaped by hindsight — a genuine per-token fix, orthogonal to ours.

Neither fix solves the other's problem. SERL-SQL's token reweighting cannot address Proposition 68, because it has no shared latent prefix to begin with. Our fix (a lower late-stage learning rate, Part IX) says nothing about *which token* inside a decoded SQL query deserves more credit than another — it never distinguishes tokens within a rollout at all. The two problems just happen to both trace back to the same root cause named in Definition 64: one scalar advantage per rollout, applied uniformly wherever it is used.

57.3 A speculative bridge (unimplemented, stated as such)

SERL-SQL's core idea — score a token using information unavailable at rollout time, then convert that into a bounded reweighting factor — is architecture-agnostic in principle. One could imagine

an analogous *hindsight latent critic*: after decoding the final SQL and learning whether it executed correctly, re-score each of the $k = 6$ latent role-steps (Part V) for how much it contributed, and reweight Corollary 69's single shared-prefix gradient unevenly across those 6 positions instead of leaving it uniform. **This is not built, not tested, and not in `latentsql/` anywhere** — it is recorded here only because SERL-SQL's mechanism is the first concrete published template for what such a fix could look like, should Part XI's open question (does GRPO reshape the subspace geometry?) turn out to need one.

57.4 The one architectural difference that is not open to interpretation

Beyond credit assignment, SERL-SQL and LatentSQL make opposite bets about *where reasoning lives at inference time*, and this part is not speculative — it follows directly from how each system is described.

	SERL-SQL	**LatentSQL (ours)**
Where does reasoning happen	explicit text, every turn decoded and readable	latent vectors, zero tokens decoded per role (Part IV)
Does inference re-execute against the database	yes — the deployed policy runs its own "think, act, and observe" loop each turn, querying the database live	no — the query-fixer role's "see an error, correct it" behaviour is distilled into weights once, during SFT (`chess_step_turns_for`, Part V); nothing is executed before the final SQL is produced
Number of decoding turns per question	multi-turn (generate, execute, revise, repeat)	one (a single forward pass through 6 latent role-slots, then decode)
Model scale reported	7B / 14B / 32B	7B backbone (student), no 32B configuration run

SERL-SQL keeps the execution-feedback loop *live*, at the cost this document has argued against throughout (Part I's cost table, Part II's decode-bottleneck argument): every deployed question pays for one or more real database round-trips and however many decoded tokens the think/act/observe turns require. LatentSQL bets the opposite way — that the *lesson* of an execution-feedback correction can be baked into weights once, at training time, so that no live loop is needed at all. Neither bet is proven superior here: SERL-SQL reports higher absolute EX (75–77% BIRD-Dev) than this project's own headline numbers (Part XIII, 57.78% on a smaller, mixed BIRD+Spider $n = 90$ set) — but at 4–5× the parameter count, on the full official BIRD dev split rather than our $n = 90$ sample, and by construction paying the live-execution cost this project spends its whole method trying to remove. Those are three simultaneous confounds, not a controlled comparison, and none of the three is resolved by the numbers alone.

Part XII
Complete map from equation to code

Equation	What it is	Where it lives
(2.1)–(2.2)	transformer, LM head, softmax	standard HF model internals
(4.1)	latent-mode input substitution	`latent.py::_latent_steps`, l.219–244
(4.2)	linear alignment $W_a = W_{\text{out}}^{\top} W_{\text{in}}$	`latent.py::build_aligner`, `mode="linear"`
(4.3)	soft-embedding alignment	`latent.py::build_aligner`, `align_soft`
$\mathcal{C}_t$ cache carry	`share_working_memory`	`latent.py::build_prefix`, l.509–535
crop no-op/undo	checkpoint-safe cache mutation	`latent.py:220,359`
(5.1)–(5.2)	role loop, stage k	`latent_sft.py::_coconut_sequence_loss`, l.104–130
`idx>=n_latent_roles :break`	Definition 38, the SQL-direct cut	`latent_sft.py:113`
(7.1)–(7.2)	masked cross-entropy, main loss	`latent_sft.py`, final `model(...,labels=...)` call, l.157–163
(7.3)–(7.4)	SQL-readiness probe	`latent_sft.py:130--150`
Proposition 52	`idx+1==sql_ready_role`	`latent_sft.py:130`
`sql_ready_active` gate	$k \geq j^{\star}$ requirement	`latent_sft.py`, l.500–506
(8.1)	LoRA	`configs/*.yaml`: `sft.lora_r=32,lora_alpha=64`
(8.2)	Adam	`latent_sft.py:407`, `torch.optim.AdamW(params ,lr=cfg.lr)`
grad-accum = batch size	Example 57	`run_chess_coconut_sql_direct.sh`, `--grad-accu m"$limit"`
optimiser reset at stage boundary	moment wipe	`run_chess_coconut_sql_direct.sh`, `rm-f"$OPTIM_ STATE"`
`clip_grad_norm_(par ams,1.0)`	gradient clipping	`latent_sft.py:597`
`gpu_settle`	CUDA-context-release wait	`run_chess_coconut_sql_direct.sh`
`LATENTSQL_REQUIRE_C UDA`	silent-CPU-fallback guard	`hf_local.py::resolve_device`
(9.1)	GRPO group advantage	`rewards.py::group_advantages`
clipped surrogate + KL	PPO-style GRPO loss	`grpo.py`, l.425–444
(9.2)	IMAD reward eq. (1)	`rewards.py::DebateReward.__call__`
w_{fmt}, l anneal	`RewardSchedule`	`rewards.py::anneal,RewardSchedule`
dense partial credit	atomic/memory terms	`rewards.py`, l.235–251
brevity term	correct-only tie-breaker	`rewards.py`, l.224–233
(6.1)/IMAD eq.(3)	difference-in-means	`caa.py`, steering-vector extraction
(6.2)	activation addition	`caa.py::steer_layer`
Definition 1	EX and ordered/unordered match	`latentsql/db/execute.py`
Part XI	GRPO's effect on agent subspaces	`05_steering.py--compare-base`, run before/after `03_grpo.py`

58 The end-to-end pipeline, as five scripts

1. `01_build_traces.py` — run the teacher MAS, produce `DebateTrace` objects, filter for consensus/correctness/execution.

2. `02_sft.py` / `02c_latent_sft.py` — stage 1 (structure learning) or the curriculum latent path of this document.

3. `03_grpo.py` — stage 2, Part X.

4. `04_eval.py` — execution accuracy on held-out BIRD/Spider, Definition 1.

5. `05_steering.py` — extract and apply the directions of Part VI.

Section 3 of the project README gives the exact commands for both the combined BIRD+Spider config and BIRD alone; `scripts/run_chess_coconut_sql_direct.sh` is the driver for the specific curriculum this document derives.

Part XIII
Results, reported honestly, including where the method loses

59 The two tests that actually define success

Every number below answers one of two questions, and they must not be conflated.

Test A (internalisation). Does the internalised student reach the same accuracy as the SFT student it started from, while its *output tokens* collapse? Reference: the SFT checkpoint. This is a test of compression.

Test B (the thesis). Does the internalised student match or beat the explicit multi-agent system *at the same model size*, at a fraction of the tokens? Reference: the explicit pipeline on the same backbone — not the larger teacher that generated the training traces.

What is explicitly not the target. A student beating the 7B teacher that generated its training data is not the claim: the teacher is a ceiling on trace quality, not a bar to clear. Recorded teacher EX on an early panel run was 64.2%; a 3B student scoring below that is not a failure of this method.

59.1 An earlier, text-mode arm: Test A never passed

Before the CHESS-step SQL-direct curriculum of Part V, an earlier arm trained IMAD's original text-mode internalisation (write the full debate, then anneal it away by length-clipping). Across three configurations of that arm, generated length moved 1024 → 922 tokens — essentially flat. Three causes were diagnosed and are recorded for completeness, since they motivate why the length-clip mechanism was abandoned in favour of the architectural cut of Definition 38:

1. the length clip l was annealed *below* the natural trace length from the first step, so the pressure to shorten was present from step 1 with no headroom to explore first;
2. `max_new_tokens` sat below l_{start}, so 16 of 24 logged steps had every rollout truncated to an identical length — nothing left to reward for being shorter (a direct instance of Corollary 65);
3. only 24 GRPO steps were run; IMAD-scale internalisation used hundreds.

This is why the project moved to the architectural approach of Part V: rather than *annealing away* verbalisation with a reward schedule, the SQL-direct curriculum *deletes the option to verbalise* by construction — Definition 38's `break` never gives later roles any text to write. This sidesteps causes 1–2 entirely, at the cost of introducing the saturated-objective failure mode of Part IX. Neither approach is free.

60 Test B on the earlier arm, before the architectural change

reference	EX	output tokens
CHESS @ 3B (the bar)	48.89%	1009
ours, SFT-only @ 3B	52.22%	1242

Half-passed: accuracy clears the bar (+3.3 points), tokens do not (1242 vs. 1009) — and the token axis is the entire point of the method, so this arm did not establish the thesis on its own.

61 The CHESS-step SQL-direct curriculum: the number that matters

This is the architecture Parts V–X derive, on the 7B backbone, $n = 90$ held-out BIRD+Spider mix, six latent roles, $m = 2$ latent steps per role (12 latent forward passes before decoding):

run	EX	bird	spider	avg out tokens	unparseable
Previous full-text SFT student	56.67%	31.11%	82.22%	1339.3	–
Curriculum, stage 6 (original)	52.22%	33.33%	71.11%	67.5	4
Curriculum, stage 6, **retrained**	**57.78%**	**37.78%**	**77.78%**	**61.2**	**0**
Curriculum, stage 6 + GRPO (from original)	**58.89%**	37.78%	**80.00%**	61.9	0

Output tokens fell **95**% (1339.3 → 61.2–67.5) while accuracy on the retrained/GRPO rows *exceeded* the full-text SFT student's bird EX (37.78% vs. 31.11%), though bird still trails the collapsed-curriculum row on overall EX by less than a point once the repair is applied. This is the strongest evidence in the project that the compression Test A asks for and the accuracy Test B asks for can be had *together* — on this later architecture, unlike the earlier text-mode arm above.

62 Cost

	greedy, latent path	explicit multi-agent (measured elsewhere in this doc)
Output tokens/question	61–68	≈ 1339
Latency/sample	2.07–2.21 s	(multiple sequential LLM calls)
Peak GPU	15.9 GB	–

62.1 Speculative decoding: measured, and a dead end on this path

Speculative decoding drafts several tokens with a small model and verifies them in one pass of the target, provably preserving the target's exact output distribution at greedy temperature (verification is lossless). It was measured at three draft sizes on the retrained-lineage GRPO checkpoint:

draft	EX	latency/sample	total	vs. greedy	acceptance length
none (greedy)	58.89%	2.09 s	188 s	1.00×	–
Coder-0.5B bf16	57.78%	2.99 s	269 s	0.70×	2.26
Coder-3B bf16	57.78%	3.39 s	305 s	0.62×	2.37
Coder-7B 4bit	57.78%	4.26 s	383 s	0.49×	2.34

Speculation is slower at every draft size measured, and the reason is structural rather than a matter of tuning: acceptance length stays flat (2.26–2.37) across a 14× range of draft size, while wall-clock improves only 1.42× over that same range — fixed per-round overhead dominates,

not draft quality. The draft model cannot see the target's latent prefix (it conditions on the plain-text prompt), so it must re-prefill the whole ≈ 1570-token prompt to cover only ≈ 62 generated tokens — speculation needs long generations to amortise its overhead, and this architecture's entire point is emitting few tokens. **Practical guidance: use greedy decoding for the latent path.**

63 Smoke-test coverage (verifies mechanism, not accuracy)

Two tiers, run CPU-only, no GPU, no data, no API key. Tier 1 (standard library only) builds two SQLite databases, runs the teacher and all four baselines, and checks trace format and every reward term. Tier 2 (needs torch) loads a tiny model and runs one real step of SFT, one GRPO step with a genuine execution-based reward, latent generation through all three aligners (Section 16), latent working memory, and steering-vector extraction and hooking. All 28 smoke checks and all 25 unit tests pass.

> Tier 2 verifies shapes, masks, caches, hooks, and schedules — *not* model quality. A tiny model cannot write correct SQL; the point is only that nothing in the mechanism is broken before a GPU-week is spent on it.

Part XIV
What is not established

Honesty about limits is part of the mathematics of this document, not an afterthought, so it gets its own Part rather than a footnote.

64 Required ablations, not yet run

Ablation	How to run it	The objection it answers
Full trace vs. final answer only	`02_sft.py --consensus-only`	is the debate transcript doing anything, or only the final answers (DebateGPT, IMAD Table 1)?
Debate panel vs. one structured format	`mas.agents: [query_plan]`	is a single execution-plan chain-of-thought already enough?
GRPO from base, no teacher	`03_grpo.py --from-examples bird --sft /nonexistent`	is the multi-agent teacher earning its cost over a pure execution-reward RL run?

The third is uncomfortable and must be reported even if unflattering: published execution-reward-only pipelines reach 68.9% BIRD dev (Arctic-Text2SQL-R1-7B) and 67.1% (SLM-SQL-1.5B) without any teacher at all. The honest external bar for this project is those numbers, not the explicit CHESS baseline.

65 The attribution gap in the stage-4 repair

Restated from Section 43 because it is the single largest open question this document leaves: two knobs (learning rate, probe role) moved together in the retrain. The measured $\mathcal{L}^{\text{probe}} \sim 10^{-4}$ throughout (Table 1) makes the learning-rate explanation far more likely than the probe explanation, but the clean experiment — $\eta = 10^{-5}$ with `SQL_READY_WEIGHT`$= 0$ — has not been run.

66 What Part VI does not establish for this system

Theorem 43 and IMAD's steering result establish that internalised multi-agent debate *can* produce separable per-agent subspaces, in IMAD's own three-persona arithmetic setting. Whether the six CHESS-role latent slots of this project occupy separable subspaces has not been measured: the extraction code (`caa.py`) exists and applies unchanged, but `05_steering.py` has not been run against the CHESS-step student.

67 What Part XI does not establish

Restated from that Part's closing table because it is the largest open question the GRPO discussion leaves: Proposition 68 and Corollary 69 prove a structural limit on what the latent-path GRPO

gradient can use as signal *within one step*; they do not show what happens to the agent-subspace geometry *across* the full GRPO run reported in Part X. That requires running `05_steering.py` once on the pre-GRPO stage-6 adapter and once on the post-GRPO adapter and comparing steering AUC — the same measurement IMAD performed for SFT, never yet performed here for RL. Separately, whether the CHESS-role subspaces are linearly separable *at all*, before any RL stage, is itself unmeasured (previous section) and is a precondition for the GRPO question to even be well-posed.

68 What Part III's capacity argument does not establish

Restated from Section 13.2: the DPI argument bounds what a *separate-call* pipeline can transmit and shows the latent channel's theoretical ceiling is far higher. It does not show the model's training actually fills that ceiling with independent, useful reasoning rather than near-duplicate copies of one state across the 12 latent positions. That question is empirical and belongs to whatever a future subspace-separability or probing measurement would answer.

69 What Part X does not establish

Section X's closing box already states the central caveat: GRPO's +6.67 EX and the curriculum repair's +5.56 EX were measured from *different* starting checkpoints and must not be summed. GRPO from the repaired stage-6 adapter — the experiment that would tell us whether the two gains compound, are redundant, or interact — has not been run.

70 Infrastructure and scale limits

- The teacher's own quality bounds the student's: with a local 7B teacher, traces are weaker than a GPT-4-class teacher would produce. The execution-based filter discards wrong traces but cannot manufacture correct ones.
- GRPO rollouts are the throughput bottleneck – group_size × prompts generations per step, each executed against a real database; the `ExecutionOracle` cache is what makes this tractable at all.
- `latent` mode decodes token-by-token in Python with no KV-cache fast path through `generate()`, so it is slower per decoded token than plain text mode even though it emits far fewer tokens overall.
- The mock LLM backend's numbers are plumbing checks only, never cited as results.
- Every result in Part XIII comes from one fixed $n = 90$ held-out mix and a rolling $n = 30$ stage probe. A 30-question probe has a standard error on the order of 9 percentage points — Section 9's own finding that the stage-6 probe was flat while the full set moved +5.56 points is a direct demonstration of that noise floor, not an anomaly.

Part XV

Every formula in one place

Level 0 — the model as a function of vectors

$$H_t = \text{Transformer}(E_t), \qquad h_t = H_t[t,:], \qquad p(x_{t+1} \mid x_{\le t}) = \text{softmax}(W_{\text{out}} h_t)$$

Level 1 — the decode bottleneck

$$I(H;T) \ge I(H;H') \quad \text{(DPI)}, \qquad I(H;T) \le L \log_2 |V| \quad \text{(ceiling)}$$

Level 2 — the continuous thought

$$\text{latent mode: next input} = \text{align}(h_t), \qquad \text{align}_{\text{soft}}(h) = \sum_{v \in \text{top}_k(W_{\text{out}} h)} \pi_v W_{\text{in}}[v,:]$$

Level 3 — the role chain and curriculum

$$\text{stage } k: \quad c \,\|\, \text{tag}(r_1)\, \bullet^m \,\|\, \cdots \,\|\, \text{tag}(r_k)\, \bullet^m \,\|\, \texttt{<|consensus|>} \,\|\, y^\star$$

Level 4 — difference-in-means

$$d = \mu_+ - \mu_-, \qquad \hat{d} = d/\|d\|, \qquad w^\star = \arg\max_{\|w\|=1} J(w) = \hat{d} \text{ (under isotropic noise)}$$

Level 5 — the training loss

$$\mathcal{L}_k^{\text{main}} = -\frac{1}{n} \sum_{i=1}^{n} \log p_\theta(g_i \mid \Phi_k, g_{<i}), \qquad \mathcal{L}_k = \mathcal{L}_k^{\text{main}} + \lambda\, \mathcal{L}^{\text{probe}}$$

Level 6 — the optimiser

$$\theta_t = \theta_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \qquad \text{step 1 after reset: } \Delta\theta \approx -\eta\, \text{sign}(g)$$

Level 7 — LoRA

$$W = W_0 + \frac{\alpha}{r} BA$$

Level 8 — the RL objective

$$\nabla_\theta J(\theta) = \mathbb{E}_{y \sim \pi_\theta}\big[\nabla_\theta \log \pi_\theta(y \mid x) R(x,y)\big], \qquad A_i = \frac{r_i - \bar{r}}{s}$$

Level 9 — the reward

$$r(x,y) = w_{\text{fmt}} R_{\text{fmt}} + w_{\text{clip}} R(y;l), \qquad R(y;l) = \mathbb{1}[y^\star \in \text{clip}(y,l)]$$

Level 10 — the metric

$$\text{EX}(y, y^\star, D) = \mathbb{1}\big[\text{exec}(y,D) = \text{exec}(y^\star, D)\big]$$

71 The one-paragraph version

A multi-agent Text-to-SQL pipeline works but forces every intermediate result through a hard bottleneck: writing it as text, which a proven inequality bounds above by $L \log_2 |V|$ bits regardless of what the state actually held. LatentSQL trades that bottleneck for a continuous one — a hidden state fed back as the next input, carrying a strictly larger channel and never re-encoded — and teaches a single small model to run through six such states, one per pipeline role, via a difference-of-means-separable internal geometry that IMAD shows internalisation actually creates. Training that curriculum with only the final SQL supervised is workable but has one sharp failure mode, proven and then observed: once a stage's objective saturates, Adam's scale-free update still takes a full step in whatever direction a near-zero, noisy gradient points, and with no penalty for reverting to the frozen base model's older habits, a mid-curriculum stage can silently collapse while its own training loss looks best of all. Lowering the late-stage learning rate repaired it, recovering +5.56 points of execution accuracy and eliminating every unparseable prediction on the full held-out set, independent of and complementary to the further +6.67 points GRPO adds through reinforcement learning proper. Neither lever, nor the architecture as a whole, is a free lunch: the honest external bar is stated, the untested ablations are named, and the one measurement that would separate "the learning rate fixed it" from "the probe fixed it" has not yet been run.